

From Context to Skills: Can Language Models Learn from Context Skillfully?

Shuzheng Si^{*♠◇}, Haozhe Zhao^{*♠}, Yu Lei^{*◇}, Qingyi Wang^{*★}, Dingwei Chen[♡]
 Zhitong Wang[♠], Zhenhailong Wang[♠], Kangyang Luo[♠], Zheng Wang[♠]
 Gang Chen[◇], Fanchao Qi[◇], Minjia Zhang[♠], and Maosong Sun[♠]
[♠]THU [◇]DeepLang AI [♠]UIUC [★]FDU [♡]CUHK
 [S1s-Z/Ctx2Skill](#) * Equal Contribution.

Abstract

Many real-world tasks require language models (LMs) to reason over complex contexts that exceed their parametric knowledge. This calls for *context learning*, where LMs directly learn relevant knowledge from the given context. An intuitive solution is inference-time skill augmentation: extracting the corresponding rules and procedures from *context* into explicit, natural-language *skills*. However, constructing such skills for context learning scenarios faces two fundamental challenges: the prohibitive cost of manual skill annotation for long, technically dense contexts, and the lack of external feedback for automated skill construction. In this paper, we propose **Ctx2Skill**, a self-evolving framework that autonomously discovers, refines, and selects context-specific skills without human supervision or external feedback. At its core, a multi-agent self-play loop has a *Challenger* that generates probing tasks and rubrics, a *Reasoner* that attempts to solve them guided by an evolving skill set, and a neutral *Judge* that provides binary feedback. Crucially, both the *Challenger* and the *Reasoner* evolve through accumulated natural-language skills: dedicated *Proposer* and *Generator* agents analyze failure cases and synthesize them into targeted skill updates for both sides, enabling automated skill discovery and refinement. To prevent adversarial collapse caused by increasingly extreme task generation and over-specialized skill accumulation, we further introduce a *Cross-time Replay* mechanism that identifies the skill set achieving the best balance across representative cases for the *Reasoner* side, ensuring robust and generalizable skill evolution. The resulting skills can be plugged into any language model to obtain better context learning capability. Evaluated on four context learning tasks from CL-bench, **Ctx2Skill** consistently improves solving rates across backbone models, e.g., lifting GPT-4.1 from 11.1% to 16.5%, GPT-5.1 from 21.2% to 25.8%.

1 Introduction

Current language models (LMs) have achieved impressive performance on problems whose relevant knowledge was present during large-scale pre-training [5, 27, 39], such as competition-level mathematical problems [14] and competitive programming challenges [17]. However, many real-world tasks require LMs to learn from complex contexts and leverage new knowledge rather than parametric knowledge to reason and solve them effectively [26, 36, 47]. Previous works refer to this capability as *context learning* [12]. Effective context learning enables models to reason beyond their pre-trained knowledge and solve complex, domain-specific tasks by learning directly from rich contextual information, much as humans do. For example, it allows LMs to rapidly make use of previously unseen product documentation to generate step-by-step operational procedures or troubleshoot issues.

Despite the central role of context learning in solving real-world tasks, it has been largely overlooked in current research [16]. A key obstacle is the diversity of real-world contexts: contextual knowledge can be conveyed through books, experimental data, and search results, among others, making context

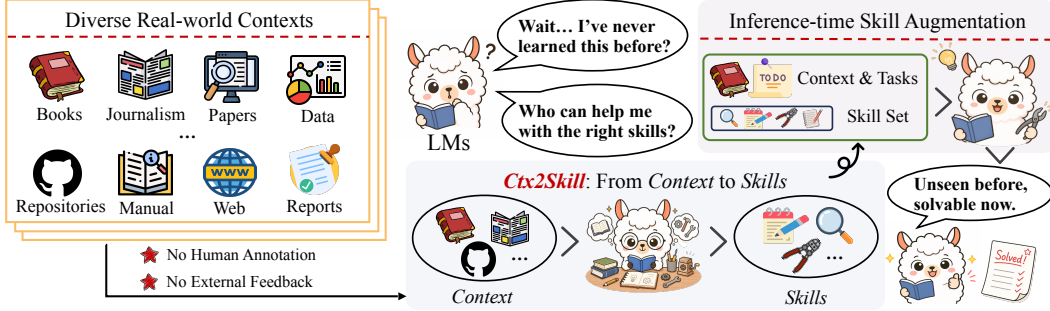


Figure 1: **The illustration of *Ctx2Skill*.** It’s designed to extract rules and procedures from *context* into natural-language *skills* without human annotation and external feedback.

learning fundamentally difficult. An intuitive paradigm is inference-time skill augmentation [3], i.e., extracting rules and procedures from *context* into natural-language *skills* that encode reusable procedural knowledge for LMs. This approach has proven effective in many agent tasks [42, 49, 37], e.g., coding agents [24]. However, this paradigm faces two fundamental challenges in context learning scenarios. Specifically, (1) **Prohibitive Cost for Manual Skill Annotation**: Existing agent skill libraries are predominantly constructed through human annotation [18, 20, 41]. However, this paradigm breaks down in context learning scenarios, where contexts are long, technically dense, and domain-specific. Curating high-quality skills requires annotators to fully internalize complex, multi-section documents, a process that is both cognitively demanding and economically infeasible, rendering human-curated skill construction impractical at scale. (2) **Lack of External Feedback for Automated Skill Construction**: Unlike verifiable tasks such as coding or mathematical reasoning [7, 31], where skill quality can be assessed via execution feedback or ground-truth comparison, context learning tasks lack automatic feedback to evaluate whether extracted skills faithfully and completely capture context-specific knowledge. Given only the context, there is no external feedback signal to tell whether a generated skill is useful or whether critical knowledge has been omitted. This feedback-free nature renders automated skill construction pipelines inapplicable [1, 44, 46]. *Together, these challenges highlight the need for an automated framework that discovers skills from the context without relying on either human annotation or external feedback signals.*

In this paper, we propose *Ctx2Skill* shown in Figure 1, which is designed to autonomously discover, refine, and select skills directly from complex contexts via skill-optimized self-play, requiring neither human annotation nor external feedback. At the core of *Ctx2Skill* is a multi-agent self-play loop [10, 22, 48, 21] comprising two competing but co-evolving roles: a *Challenger* agent, which generates a batch of tasks and associated rubrics [15] based on context and its skill set, aiming to probe deep contextual understanding, and a *Reasoner* agent, which reads the context and attempts to solve these tasks guided by its current skill set. The core idea of *Ctx2Skill* is that the two agents co-evolve by iteratively updating their respective skill sets rather than model parameters. Specifically, a neutral *Judge* agent evaluates the *Reasoner*’s responses against each rubric, producing feedback on whether it passes. Crucially, both the *Challenger* and *Reasoner* evolve through accumulated natural-language skills: failed cases are routed to dedicated *Proposer* and *Generator* agents on the *Reasoner* side to diagnose missing contextual knowledge and update skills accordingly, while easily solved cases are routed to the *Challenger* side to strengthen its task and rubric generation strategies, ensuring sustained adversarial pressure. The updated skills are carried forward into the next iteration, allowing both sides to continuously co-evolve through failure-driven textual feedback without any parameter updates. Furthermore, we identify a key risk in this multi-agent self-play loop, namely *adversarial collapse* phenomenon: as iterations progress, the *Challenger* may generate increasingly extreme tasks while the *Reasoner*’s skills over-specialize to these pathological cases, resulting in redundant skill accumulation that degrades generalization. To address this, we further propose a *Cross-time Replay* mechanism that selects the skill set that best balances performance across representative cases, ensuring robust and generalizable skill evolution. At inference time, the resulting context-specific *Reasoner* skills can be plugged into any LM to enhance its context learning capability on arbitrary tasks over the same context, including those unseen during the self-play loop.

We evaluate our *Ctx2Skill* on four context learning tasks from CL-Bench [12], including Domain Knowledge Reasoning, Rule System Application, Procedural Task Execution, and Empirical Discovery & Simulation. *Ctx2Skill* consistently improves solving rate across all four categories and

multiple backbone models: it raises GPT-4.1 [29] from 11.1% to 16.5%, GPT-5.1 [27] from 21.2% to 25.8%, and GPT-5.2 [27] from 18.2% to 21.4%, demonstrating that autonomously discovered context-specific skills provide substantial and generalizable gains.

2 Related Work

Context Learning. Current language models excel at tasks whose relevant knowledge was present during pre-training [9, 27, 33, 34]. However, many real-world tasks require models to learn from complex contexts and leverage new knowledge to reason effectively, a capability termed *context learning* [12]. This context learning capability is essential in practice: a physician consults newly released clinical guidelines to adjust treatment, an engineer reads documentation to execute procedures, and a researcher induces patterns from experimental data to form hypotheses. However, prior work finds that LMs have yet to master this capability [12]. The core difficulty is that contextual knowledge spans diverse formats, from books to experimental datasets, and the relevant rules and procedures are often embedded implicitly, requiring deep comprehension rather than surface-level retrieval. *Ctx2Skill* directly targets this challenge by autonomously discovering reusable, context-specific skills from the contexts, equipping any LM to better learn from and reason over complex contexts.

Skills for LMs. Skills are natural-language modules that encode reusable procedural knowledge to augment LMs at inference time [2, 3], and skill augmentation has been widely validated across agent tasks [32], e.g., coding [24] and web navigation [42]. Early skill libraries are predominantly constructed through human annotation [18, 19, 20, 41], which is effective but does not scale, especially for context learning scenarios where contexts are long, technically dense, and domain-specific. Recent work has shifted toward automated skill construction. AutoSkill [44] extracts reusable behaviors from interaction traces as lifelong versioned artifacts, and AutoRefine [30] converts agent trajectories into reusable expertise. CoEvoSkills [45] generates multi-file skill packages and refines them through generator–verifier co-evolution, while EvoSkill [1] performs failure-driven refinement into structured skill folders. SkillX [40] distills the agent trajectories into a hierarchical skill knowledge base and iteratively refines them through execution feedback. However, these methods rely on external feedback signals [25], such as execution feedback, ground-truth comparison, or task-completion rewards, to evaluate and improve skill quality, which are not available in context learning scenarios where there is no automatic feedback. A separate line of work tries to internalize skills into model parameters. SKILL0 [23] uses in-context reinforcement learning (RL) to absorb skills, and SkillRL [43] builds a hierarchical skill bank via RL-guided distillation from teacher trajectories. These approaches require parameter access, making them inapplicable to closed-source models, and sacrificing the interpretability of natural-language skill documents. In this paper, we propose *Ctx2Skill*, a self-evolving framework designed to directly discover and evolve skills from complex contexts alone, which requires no human annotation, no external feedback, and no parameter updates.

3 Methodology

3.1 Overview

Context learning tasks [12] require LMs to leverage contextual knowledge from long, technically dense contexts to reason and solve tasks effectively. An intuitive paradigm for context learning tasks is inference-time skill augmentation, i.e., extracting the corresponding rules and procedures from *context* into natural-language *skills*. Turning this idea into a working framework, however, faces two difficulties: (1) prohibitive cost for manual skill annotation and (2) lack of external feedback for automated skill construction. In this paper, we introduce *Ctx2Skill* shown in Figure 2, a self-evolving framework that autonomously discovers, refines, and selects context-specific skills without human supervision or external feedback. At its core, a *skill-optimized self-play* loop (§ 3.3) co-evolves two competing sides: a *Challenger* probes the context with tasks and rubrics guided by its own skill set, a *Reasoner* solves them under its current skill set, and a neutral *Judge* returns binary verdicts that route each outcome to a side-specific *Proposer–Generator* pair, which diagnoses the exposed weakness and edits that side’s skills accordingly, so that both sides co-evolve without human annotation or external feedback. Furthermore, a key risk in this self-play framework is *adversarial collapse*: the *Challenger* may generate increasingly extreme tasks while the *Reasoner*’s skills over-specialize to these pathological cases, resulting in redundant skill accumulation that degrades generalization. To address this, we further propose a *Cross-Time Replay* mechanism (§ 3.4) that selects the skill set

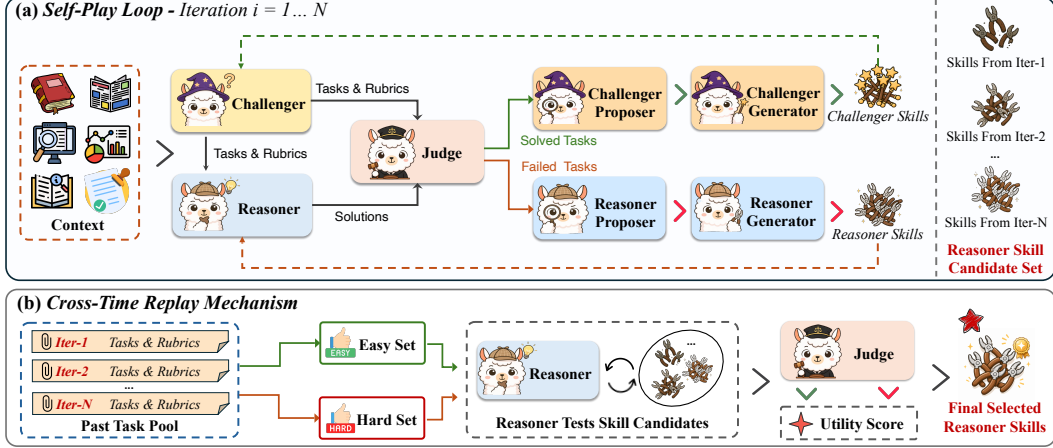


Figure 2: **The overview of Ctx2Skill.** (a) In the self-play loop, the *Challenger* generates tasks and rubrics, the *Reasoner* tries to solve them, and the *Judge* routes outcomes to update *Challenger* and *Reasoner* skills. (b) Cross-Time Replay Mechanism re-evaluates historical *Reasoner* skill candidates on representative tasks, selecting the most balanced skill set for unseen downstream tasks.

that best balances performance across representative cases, ensuring robust and generalizable skill evolution. In this way, *Ctx2Skill* autonomously constructs context-specific skills that can be plugged into any LM at inference time to enhance context learning capability without parameter updates.

3.2 Problem Formulation

A context learning task [12] consists of a given context C whose content may lie outside the LM’s pretraining corpus, a set of tasks $\mathcal{T} = \{t_j\}$ whose correct answers depend on C , and a set of binary rubrics $\mathcal{R}_j = \{r_{j,k}\}$. Given an answer a_j produced by an LM π , the task is considered solved only when every rubric passes, we define a solving indicator $y_j(\pi; C)$ for task t_j as:

$$y_j(\pi; C) = \prod_k \mathbb{I}[r_{j,k}(a_j) = \text{pass}], \quad a_j \sim \pi(\cdot | C, t_j). \quad (1)$$

Our goal is to enable the LM π to solve tasks over a previously unseen context C without any parameter update or external feedback. To this end, we introduce a natural-language *skill set* $\mathcal{S}$, a short Markdown file pre-pended to the system prompt of the LM at inference time:

$$a_j \sim \pi(\cdot | \mathcal{S}, C, t_j). \quad (2)$$

During the skills construction of our proposed *Ctx2Skill*, $\mathcal{S}$ is instantiated as two separate skill sets: $\mathcal{S}^R$ for the *Reasoner* agent and $\mathcal{S}^C$ for the *Challenger* agent. At inference time, only the *Reasoner*’s skill set $\mathcal{S}^R$ is deployed with an LM for context-specific tasks evaluation.

3.3 Ctx2Skill Framework

Building a skill set for an unseen context C faces two obstacles: human annotation is costly to read C and write down its contextual knowledge, and no external feedback signal can tell us whether a proposed skill is faithful and useful to C . To address both, *Ctx2Skill* runs a multi-agent self-play loop that autonomously discovers and refines context-specific skills from context C alone.

The core idea of *Ctx2Skill* is to co-evolve two skill sets, skill set $\mathcal{S}^R$ for the *Reasoner* and skill set $\mathcal{S}^C$ for the *Challenger*, through N iterations of failure-driven textual edits, so that the *Reasoner* gradually accumulates the contextual knowledge of the context C while the *Challenger* keeps probing aspects not yet mastered. Five frozen-LM agent roles carry out this self-play loop. At iteration i , given the context C and skill set $\mathcal{S}_{i-1}^C$, the *Challenger* produces a batch of tasks with corresponding rubrics; the *Reasoner*, guided by context C and its skills $\mathcal{S}_{i-1}^R$, attempts each task; the neutral *Judge* returns per-task binary verdicts and partitions the batch into failed and solved tasks. Each group is then routed to its corresponding side through a *Proposer-Generator* pair: failed cases flow to the *Reasoner* side, where the *Proposer* diagnoses which contextual knowledge is missing and the *Generator* updates $\mathcal{S}^R$

accordingly; solved cases flow to the *Challenger* side, where the corresponding pair tightens $\mathcal{S}^C$ so that the next iteration sustains adversarial pressure. The updated $\mathcal{S}_i^R$ and $\mathcal{S}_i^C$ are carried into iteration $i+1$, and no prompt is ever conditioned on the opposing side’s skill set, keeping the two sides strictly adversarial. We now define the five agent roles as follows at the iteration i :

- **Challenger**: Given the context C and its current skill set $\mathcal{S}_{i-1}^C$, the *Challenger* produces a batch of M tasks $\{t_m\}_{m=1}^M$, each equipped with corresponding rubrics $\mathcal{R}_m$. The rubrics are designed so that a correct answer requires inducing the rules of C rather than paraphrasing surface spans. By conditioning on its skills $\mathcal{S}_{i-1}^C$, the *Challenger* adapts its probing strategy across iterations, sustaining adversarial pressure as the *Reasoner* improves.
- **Reasoner**: Given C , a task t_m , and its current skill set $\mathcal{S}_{i-1}^R$, the *Reasoner* produces an answer a_m . While the raw context C contains all necessary knowledge, it is often long and technically dense; the skill set $\mathcal{S}_{i-1}^R$ distills the most relevant rules and procedures from C into a concise, structured form, allowing the *Reasoner* to directly apply contextual knowledge that would otherwise need to be extracted from scratch at every task.
- **Judge**: Given a task and rubrics generated from the *Challenger*, and the generated answer from the *Reasoner*, the *Judge* returns a per-rubric binary verdict $z_{m,k} = \mathbb{I}[r_{m,k}(a_m) = \text{pass}]$ and a solving indicator $y_m = \prod_k z_{m,k}$. The *Judge* is a strict evaluator: the task is considered solved if and only if the answer satisfies all associated rubrics.
- **Proposer (one per side)**: The *Judge*’s binary verdicts identify which tasks failed or solved, but not why. The *Proposer* therefore collects all routed cases of its side, where each case comprises the task and rubrics from the *Challenger*, and the *Reasoner*’s answer, and jointly analyzes them against the current skill set. Rather than diagnosing each case in isolation, the *Proposer* synthesizes common failure or success patterns across cases into a high-level diagnosis specifying an action (add or merge), the target skill name, a description, and a justification. The **Reasoner Proposer** is designed to take the failed cases $\mathcal{F}_i$ together with $\mathcal{S}_{i-1}^R$, and diagnose which contextual knowledge is missing or misrepresented by examining both the incorrect answers and the rubrics they violated. On the other hand, the **Challenger Proposer** takes the easily solved cases $\mathcal{P}_i$ together with $\mathcal{S}_{i-1}^C$, and identifies gaps in the current task and rubric generation strategy by inspecting how the *Reasoner* solved them with its existing skills.
- **Generator (one per side)**: The *Proposer*’s output is a high-level diagnosis describing what should change and why, but not the concrete skill content itself. The *Generator* materializes this diagnosis into an actual skill set. Given the diagnosis and the current skill set, it returns a complete replacement skill set that adds or merges entries while preserving every unrelated entry. The **Reasoner Generator** incorporates the diagnosed missing contextual knowledge into $\mathcal{S}_{i-1}^R$ to produce the new skill set $\mathcal{S}_i^R$. The **Challenger Generator** tightens $\mathcal{S}_{i-1}^C$ so that the next iteration probes the *Reasoner*’s remaining weaknesses, producing skills $\mathcal{S}_i^C$.

To summarize, at iteration i , the *Challenger* first produces a batch of tasks and rubrics $\{(t_m, \mathcal{R}_m)\}_{m=1}^M$ based on the context C ; the *Reasoner* attempts to answer each t_m ; the *Judge* evaluates every rubric and partitions the batch into failed cases $\mathcal{F}_i = \{m : y_m = 0\}$ and solved cases $\mathcal{P}_i = \{m : y_m = 1\}$ based on the solving indicator y_m . Each group is then routed to its corresponding side:

- The *Reasoner Proposer* jointly analyzes all failed cases $\mathcal{F}_i$ against $\mathcal{S}_{i-1}^R$ and synthesizes a high-level diagnosis; the *Reasoner Generator* then materializes it into $\mathcal{S}_i^R$.
- The *Challenger Proposer* jointly analyzes all solved cases $\mathcal{P}_i$ against $\mathcal{S}_{i-1}^C$ and synthesizes a high-level diagnosis; the *Challenger Generator* then materializes it into $\mathcal{S}_i^C$.

The updated skill sets $\mathcal{S}_i^R$ and $\mathcal{S}_i^C$ are carried forward into iteration $i+1$. No prompt is ever conditioned on the opposing side’s skill set, keeping the two sides strictly adversarial.

3.4 Cross-Time Replay Mechanism

The framework of *Ctx2Skill* detailed in § 3.3 intentionally strengthens the *Challenger* across iterations through updates to $\mathcal{S}^C$, sustaining adversarial pressure as the *Reasoner* improves. However, this design introduces an inherent tension that we term *adversarial collapse*. As iterations progress, the *Challenger* generates increasingly extreme tasks that concentrate on the *Reasoner*’s residual weak-

nesses, gradually deviating from the representative knowledge of the context C . Since the *Reasoner*’s skill updates are failure-driven, its skills over-specialize to these pathological cases, resulting in redundant skill accumulation that degrades generalization. Moreover, this collapse is undetectable within the loop: each iteration’s *Judge* only evaluates the *Challenger*’s newly generated tasks, providing no signal on whether contextual knowledge mastered in earlier iterations has been corrupted by subsequent edits. Returning the last iteration *Reasoner*’s skills $\mathcal{S}_N^R$ unconditionally is therefore unreliable; we instead select the most generalizable skill set among candidates $\{\mathcal{S}_1^R, \dots, \mathcal{S}_N^R\}$.

To address this, we propose a *cross-time replay* mechanism that selects the most generalizable skill set from the candidates $\{\mathcal{S}_1^R, \dots, \mathcal{S}_N^R\}$ using representative probes collected during self-play itself. Two small probe sets are curated incrementally without any external supervision: at each iteration i , the failed task with the lowest rubric pass rate is added to the hard probe set $\mathcal{Q}^h$, and the solved task with the fewest rubrics is added to the easy probe set $\mathcal{Q}^e$, capturing the hardest failure and the easiest success observed at each iteration. Once the self-play loop terminates, the *Reasoner* π^R re-answers every task in both probe sets under each candidate skill set $\mathcal{S}_i^R$, and the *Judge* re-evaluates every rubric, yielding the solving indicator y_q (i.e., whether all rubrics pass) for each probe task q . We define the Laplace-smoothed [13] solving rates $\rho^h(i)$ and $\rho^e(i)$ as the proportion of tasks solved in the hard and easy probe sets under the context C and skills $\mathcal{S}_i^R$, respectively:

$$\rho^h(i) = \frac{\sum_{q \in \mathcal{Q}^h} y_q(\pi^R; C, \mathcal{S}_i^R) + 1}{|\mathcal{Q}^h| + 1}, \quad \rho^e(i) = \frac{\sum_{q \in \mathcal{Q}^e} y_q(\pi^R; C, \mathcal{S}_i^R) + 1}{|\mathcal{Q}^e| + 1}. \quad (3)$$

The selected skill set is the one that maximizes the product of these two solving rates:

$$\mathcal{S}_\star^R = \mathcal{S}_{i^\star}^R, \quad i^\star = \arg \max_i (\rho^h(i) \cdot \rho^e(i)). \quad (4)$$

The multiplicative form is essential: a skill set that resolves late, idiosyncratic failures at the cost of regressing on easier tasks incurs a penalty on $\rho^e(i)$ and is rejected; conversely, one that solves every easy probe but fails every hard one is rejected through $\rho^h(i)$. Meanwhile, Laplace smoothing keeps the product well-defined when a probe set is empty for a given iteration.

At inference time, the final skill set $\mathcal{S}_\star^R$ is prepended to the *Reasoner*’s system prompt for any unseen task t_u over context C , and the *Reasoner* answers conditioned on $(\mathcal{S}_\star^R, C, t_u)$. Since $\mathcal{S}_\star^R$ encodes reusable contextual knowledge of C rather than task-specific solutions, it generalizes to arbitrary tasks over the same context. Moreover, $\mathcal{S}_\star^R$ is produced once per context and reused across all $|\mathcal{T}|$ unseen tasks, so the *Ctx2Skill* cost amortizes over $|\mathcal{T}|$ with no additional cost per deployed query.

4 Experiments

In this section, we conduct experiments and analyses to show the advantages of our *Ctx2Skill*.

4.1 Settings

Evaluation. Context learning is a newly emerging research topic, aiming to require LMs to learn from unseen context and leverage new knowledge beyond what was acquired during pre-training to reason and solve tasks [12]. This goes far beyond long-context tasks [6, 35] that primarily test retrieval or reading comprehension, and in-context learning tasks [11], where models learn simple task patterns via instructions and demonstrations. We select the CL-bench [12] for our evaluation, which comprises 500 complex contexts, 1,899 tasks, and 31,607 verification rubrics, all crafted by experienced domain experts. Each task is designed such that the new content required to resolve it is contained within the corresponding context. These data are distributed across four categories: *Domain Knowledge Reasoning*, *Rule System Application*, *Procedural Task Execution*, and *Empirical Discovery & Simulation*. Scoring is strictly all-or-nothing: a task is considered solved only when every rubric passes. We report both per-category and overall solving rates. Rubric-level grading follows the original benchmark protocol, which employs GPT-5.1 as the LLM-as-a-judge verifier [50]; cross-verifier and human agreement with this verifier both exceed 90% [12].

Baselines. We first evaluate a broad set of frontier LMs, including GPT-4.1 [29], GPT-5.1 [27], GPT-5.2 [28], Claude Opus 4.5 [4], Gemini 3 Pro [8], Kimi K2.5 [38], and DeepSeek V3.2 [9]. For models that support thinking mode, we use the default reasoning effort. For skill-based methods, we

Table 1: **The main results of *Ctx2Skill***. We report the task-solving rate on the CL-bench across four task categories. The best results within the same backbone are marked in **bold**. **○** represents methods using inference time augmentation, and **○** shows inference results without skills. **Red** and **Green** show the performance gains and declines, respectively, relative to the models without skills.

Model	Overall (%)	Domain Knowledge Reasoning (%)	Rule System Application (%)	Procedural Task Execution (%)	Empirical Discovery & Simulation (%)
Frontier LMs					
○ GPT-5.1	21.1	22.4	21.0	22.8	13.6
○ Claude Opus 4.5	21.0	23.7	19.0	22.6	15.1
○ Kimi K2.5	19.2	19.1	19.4	21.3	14.4
○ GPT-5.2	18.2	19.5	18.0	19.1	12.1
○ Gemini 3 Pro	15.8	15.5	17.7	16.4	10.1
○ DeepSeek V3.2 Thinking	13.2	13.6	13.8	14.2	8.0
○ GPT-4.1	11.1	10.6	14.8	10.4	4.6
GPT-4.1-based Methods					
○ Prompting	12.3 +1.2	12.4 +1.8	12.3 -2.5	13.9 +3.5	8.2 +3.6
○ AutoSkill4Doc	13.2 +2.1	13.3 +2.7	13.1 -1.7	15.0 +4.6	8.7 +4.1
○ <i>Ctx2Skill</i>	16.5 +5.4	16.8 +6.2	17.6 +2.8	17.6 +7.2	9.7 +5.1
GPT-5.1-based Methods					
○ Prompting	22.1 +1.0	24.7 +2.3	21.1 +0.1	22.4 -0.4	15.5 +1.9
○ AutoSkill4Doc	22.7 +1.6	25.3 +2.9	21.5 +0.5	23.1 +0.3	16.0 +2.4
○ <i>Ctx2Skill</i>	25.8 +4.7	27.9 +5.5	24.9 +3.9	26.9 +4.1	19.1 +5.5
GPT-5.2-based Methods					
○ Prompting	19.1 +0.9	19.6 +0.1	18.3 +0.3	22.6 +3.5	11.1 -1.0
○ AutoSkill4Doc	19.7 +1.5	20.5 +1.0	18.8 +0.8	23.0 +3.9	11.6 -0.5
○ <i>Ctx2Skill</i>	21.4 +3.2	22.2 +2.7	20.4 +2.4	25.4 +6.3	12.6 +0.5

compare *Ctx2Skill* with two baselines: (1) *Prompting* directly prompts the LM to read the context C and generate a skill set in a single pass; the resulting skills are then prepended to LMs during inference. (2) *AutoSkill4Doc* is a variant of AutoSkill [44] designed for document-level contexts: it partitions the context C into different windows, identifies the independent skills within each window, and recombines skills across windows to produce a skill set. Both baselines use the same backbone for skill generation and inference as *Ctx2Skill*: for example, all GPT-4.1-based methods use GPT-4.1 for both skill construction and evaluation, ensuring a fair comparison.

Implementation. We run $N=5$ self-play iterations per context with $M=5$ tasks per iteration. The *Challenger*, *Reasoner*, *Proposer*, and *Generator* all use the same backbone as the corresponding series (e.g., GPT-4.1 for all four agents in the GPT-4.1-based methods), while the *Judge* uses GPT-5.1, consistent with the CL-bench evaluation protocol [12]. Due to API budget constraints, we do not explore larger N or M ; nonetheless, the most effective skill set is typically selected from early iterations (§ 4.3), and $M=5$ tasks per iteration provide enough cases for the *Proposer* to synthesize meaningful diagnostic patterns.

4.2 Results

Table 1 presents the main results on CL-bench. Context learning remains challenging for current frontier LMs: even the best-performing model, GPT-5.1, achieves only a 21.1% overall solving rate. *Ctx2Skill* consistently improves solving rates across all three backbones, raising GPT-4.1 from 11.1% to 16.5% (+5.4%), GPT-5.1 from 21.1% to 25.8% (+4.6%), and GPT-5.2 from 18.2% to 21.4% (+3.2%), outperforming both *Prompting* and *AutoSkill4Doc* by a large margin across all four categories. The gains are especially pronounced on Procedural Task Execution and Empirical Discovery & Simulation, which demand deeper procedural and inductive reasoning over the context. In contrast, the two baselines provide only modest improvements and occasionally degrade individual categories (e.g., *Prompting* decreases Rule System Application on GPT-4.1 by 2.5%), suggesting that single-pass skill extraction is insufficient for complex contextual knowledge. Notably, GPT-4.1

Table 2: **The quality evaluation of generated skills**. We use GPT-4.1 as a judge to assess skills across five dimensions: conciseness, faithfulness, clarity, effectiveness, and reusability. The best result in each dimension is marked in **bold**.

Model	Conc.	Faith.	Clar.	Eff.	Reus.	Avg.
GPT-4.1-based Methods						
Prompting	81.2	79.7	80.0	83.3	84.7	81.8
AutoSkill4Doc	81.3	81.4	92.4	88.7	87.2	86.2
<i>Ctx2Skill</i>	85.2	84.8	96.2	90.5	92.5	89.8
GPT-5.1-based Methods						
Prompting	80.7	88.5	94.1	94.6	95.3	90.7
AutoSkill4Doc	82.0	89.1	94.2	96.0	96.0	91.5
<i>Ctx2Skill</i>	82.6	93.9	98.1	96.7	96.9	93.6
GPT-5.2-based Methods						
Prompting	80.5	84.4	92.0	90.4	92.0	87.9
AutoSkill4Doc	82.2	87.0	94.0	92.2	93.0	89.7
<i>Ctx2Skill</i>	85.1	89.6	96.5	94.7	94.4	92.0

Table 3: **The analysis results of *Ctx2Skill*.** We report the task-solving rate on the CL-bench across four task categories under different ablations, variant designs, etc. The best results within each analysis block are marked in **bold**. **○** represents using inference-time skill augmentation, and **○** indicates inference results without skills.

Model	Overall (%)	Domain Knowledge Reasoning (%)	Rule System Application (%)	Procedural Task Execution (%)	Empirical Discovery & Simulation (%)
Ablation Study					
○ Base Model - GPT-4.1	11.1	10.6	14.8	10.4	4.6
○ <i>Ctx2Skill</i> - GPT-4.1	16.5	16.8	17.6	17.6	9.7
-w/o Cross-Time Replay Mechanism	14.7	15.1	14.9	16.1	9.2
-w/o Decoupling Proposer and Generator	15.9	15.9	16.7	17.4	9.7
-w/o Challenger Skills Evolving	13.8	13.9	13.9	15.7	8.7
-w/o Easy Probe Set	15.7	15.7	16.3	17.4	9.7
-w/o Hard Probe Set	15.2	15.6	15.6	16.5	9.2
-w/o Laplace Smoothing	15.5	15.7	16.0	17.2	9.2
○ Base Model - GPT-5.1	21.1	22.4	21.0	22.8	13.6
○ <i>Ctx2Skill</i> - GPT-5.1	25.8	27.9	24.9	26.9	19.1
-w/o Cross-Time Replay Mechanism	23.0	25.6	22.0	23.3	16.0
-w/o Decoupling Proposer and Generator	25.1	27.0	24.5	25.9	18.6
-w/o Challenger Skills Evolving	22.5	25.0	21.3	23.1	16.0
-w/o Easy Probe Set	24.2	26.7	23.6	23.9	18.0
-w/o Hard Probe Set	24.7	26.8	24.0	25.2	18.0
-w/o Laplace Smoothing	25.2	27.0	24.7	25.9	19.1
Variant Designs Testing					
○ Base Model - GPT-4.1	11.1	10.6	14.8	10.4	4.6
○ <i>Ctx2Skill</i> - GPT-4.1	16.5	16.8	17.6	17.6	9.7
○ Loser-Only Skill Update	16.0	15.9	16.9	17.6	9.7
○ Joint Outcome Skill Update	15.5	15.7	16.0	17.2	9.2
○ Using Additive Scoring in Eq.(4)	15.9	16.6	17.1	16.5	8.2
Effect of Cross-Time Replay Mechanism					
○ Base Model - GPT-4.1	11.1	10.6	14.8	10.4	4.6
○ <i>Ctx2Skill</i> - GPT-4.1	16.5	16.8	17.6	17.6	9.7
○ Using Skills from Iter-1	15.9	15.9	16.7	17.4	9.7
○ Using Skills from Iter-2	15.6	15.7	16.2	17.2	9.7
○ Using Skills from Iter-3	15.6	15.7	16.2	17.2	9.7
○ Using Skills from Iter-4	15.2	15.6	15.6	16.5	9.2
○ Using Skills from Iter-5	14.7	15.1	14.9	16.1	9.2
Skill Transferability Testing					
○ Base Model - GPT-4.1	11.1	10.6	14.8	10.4	4.6
○ GPT-4.1 with Skills from GPT-5.1	16.1	16.6	17.2	17.2	8.2
○ GPT-4.1 with Skills from GPT-4.1	16.5	16.8	17.6	17.6	9.7
○ Base Model - GPT-5.1	21.1	22.4	21.0	22.8	13.6
○ GPT-5.1 with Skills from GPT-4.1	23.1	24.9	22.4	24.1	16.5
○ GPT-5.1 with Skills from GPT-5.1	25.8	27.9	24.9	26.9	19.1

with *Ctx2Skill* skills (16.5%) surpasses stronger frontier models without skills, e.g., Gemini 3 Pro (15.8%), demonstrating that context-specific skills can bridge substantial capability gaps. Beyond solving rate, we assess the intrinsic quality of generated skills using GPT-4.1 as a judge across five dimensions (Table 2). *Ctx2Skill* achieves the highest average score across all three backbones, outperforming *AutoSkill4Doc* by +3.6, +2.1, and +2.3 on GPT-4.1, GPT-5.1, and GPT-5.2, with the most notable improvements in faithfulness and clarity. These results indicate that the iterative self-play loop produces skills that not only improve downstream reasoning but also present contextual knowledge in a well-structured, human-readable form amenable to inspection, editing, and reuse.

4.3 Analysis

Ablation Study. We ablate each component of *Ctx2Skill* on both the GPT-4.1 and GPT-5.1 (Table 3, Ablation Study block). Removing *Challenger* skills evolving causes the largest drop, confirming that sustained adversarial pressure is essential for the *Reasoner* to discover contextual knowledge progressively. The *Cross-Time Replay* mechanism is the second most impactful: without it, the last-iteration skills suffer from adversarial collapse. Within *Cross-Time Replay*, the hard probe set contributes more than the easy probe set, and removing Laplace smoothing also degrades performance. Merging the *Proposer* and *Generator* into a single agent yields a modest but consistent decline, supporting the decoupling of diagnosis from skill materialization. We also examine the impact across all sub-categories of CL-bench on GPT-4.1 (Figure 3). *Ctx2Skill* improves solving rates on the vast majority of sub-categories, with especially large gains on workflow orchestration (+11.8%).

Variant Designs Testing. We examine three alternative designs on GPT-4.1 (Table 3, Variant Designs Testing block). *Loser-Only Skill Update* updates only the losing side each iteration; the 0.5% drop

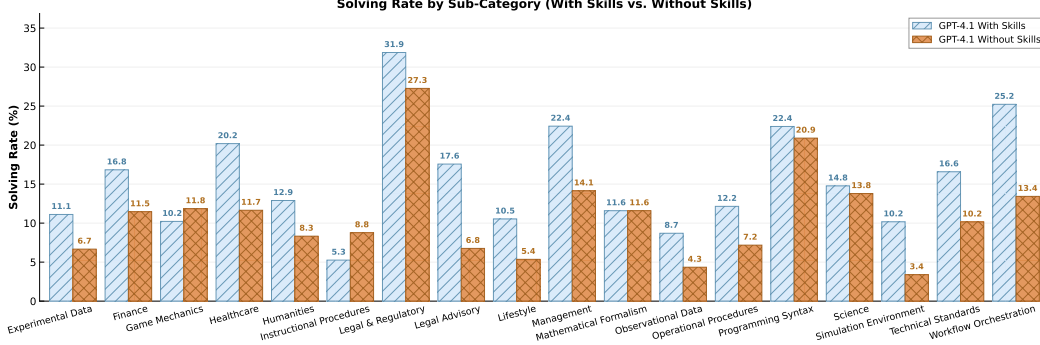


Figure 3: **Per sub-category solving rate on CL-bench.** *Ctx2Skill* improves solving rates on the vast majority of sub-categories compared with the base model without skills.

indicates that updating both sides yields more effective co-evolution. *Joint Outcome Skill Update* feeds both failed and solved cases into both sides simultaneously, allowing each side to learn from both positive and negative examples; the larger 1.0% degradation suggests that mixing outcomes dilutes the diagnostic signal. We also test *Additive Scoring*, which replaces $\rho^h(i) \cdot \rho^e(i)$ with $\rho^h(i) + \rho^e(i)$ in Eq. (4); the 0.6% drop confirms that the multiplicative form better penalizes skill sets that sacrifice easy-probe performance for hard-probe gains.

Effect of Cross-Time Replay Mechanism. We compare *Cross-Time Replay* against using a fixed iteration’s skills (Table 3, Effect of Cross-Time Replay block). On GPT-4.1, fixed-iteration performance monotonically decreases from Iter-1 (15.9%) to Iter-5 (14.7%), confirming that later iterations suffer from adversarial collapse phenomenon. *Cross-Time Replay* (16.5%) outperforms every fixed iteration, including the best one (Iter-1), by +0.6%, by adaptively selecting the most balanced skill set per context rather than committing to a single iteration across all contexts. Figure 4 corroborates this pattern: early iterations are selected most frequently across all three backbones, while a non-trivial proportion of later iterations indicates that certain contexts with more complex knowledge structures do benefit from additional self-play rounds.

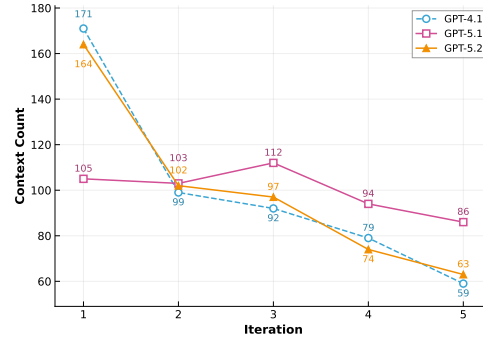


Figure 4: **Distribution of selected iterations by the Cross-Time Replay.** We report the number of contexts whose final skill set is selected from each iteration across all contexts.

Skill Transferability Testing. We investigate whether skills generated by one backbone can benefit a different one (Table 3, Skill Transferability Testing block). GPT-5.1-generated skills applied to GPT-4.1 yield 16.1%, nearly matching GPT-4.1’s own skills; conversely, GPT-4.1-generated skills applied to GPT-5.1 yield 23.1%, a +2.0% gain but notably below the +4.6% from GPT-5.1’s own skills. This asymmetry shows that stronger models produce skills that transfer well to weaker models, whereas weaker models lack the capacity to discover knowledge that stronger models can exploit.

Case Study. We conduct a case study in Appendix C to visually show the advantages of *Ctx2Skill*.

5 Conclusion

We presented *Ctx2Skill*, a self-evolving framework that autonomously discovers, refines, and selects context-specific skills from complex contexts without human annotation or external feedback. Through a skill-optimized self-play loop, a *Challenger* and a *Reasoner* co-evolve their skill sets via failure-driven textual edits, while a *Cross-Time Replay* mechanism prevents adversarial collapse by selecting the most generalizable skill set across iterations. Experiments on CL-bench demonstrate that *Ctx2Skill* consistently and substantially improves context learning performance across multiple backbone models and task categories, and that the resulting skills are transferable across models. We hope *Ctx2Skill* provides a practical and scalable paradigm for equipping language models with the ability to learn skillfully from complex, previously unseen contexts.

References

- [1] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. 2026. Evoskill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*.
- [2] Anthropic. 2025. [Equipping agents for the real world with agent skills](#).
- [3] Anthropic. 2025. [Introduction to agent skills](#).
- [4] Anthropic. 2025. [System card: Claude opus 4.5](#).
- [5] Anthropic. 2026. [System card: Claude opus 4.6](#).
- [6] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- [7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- [8] Google DeepMind. 2025. [Gemini 3 pro model card](#).
- [9] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojuan Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuaung Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shutong Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. [Deepseek-v3 technical report](#). *Preprint*, arXiv:2412.19437.
- [10] Anthony DiGiovanni and Ethan C. Zell. 2021. [Survey of self-play in reinforcement learning](#). *Preprint*, arXiv:2107.02850.
- [11] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, Lei Li, and Zhifang Sui. 2023. [A survey on in-context learning](#). *Preprint*, arXiv:2301.00234.
- [12] Shihan Dou, Ming Zhang, Zhangyue Yin, Chenhao Huang, Yujiong Shen, Junzhe Wang, Jiayi Chen, Yuchen Ni, Junjie Ye, Cheng Zhang, Huaibing Xie, Jianglu Hu, Shaolei Wang, Weichao Wang, Yanling Xiao, Yiting Liu, Zenan Xu, Zhen Guo, Pluto Zhou, Tao Gui, Zuxuan Wu,

- Xipeng Qiu, Qi Zhang, Xuanjing Huang, Yu-Gang Jiang, Di Wang, and Shunyu Yao. 2026. [CL-Bench: A benchmark for context learning](#). *arXiv preprint arXiv:2602.03587*.
- [13] David A Field. 1988. Laplacian smoothing and delaunay triangulations. *Communications in applied numerical methods*, 4(6):709–712.
 - [14] Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, Zhengyang Tang, Benyou Wang, Daoguang Zan, Shanghaoran Quan, Ge Zhang, Lei Sha, Yichang Zhang, Xuancheng Ren, Tianyu Liu, and Baobao Chang. 2025. [Omni-MATH: A universal olympiad level mathematic benchmark for large language models](#). In *The Thirteenth International Conference on Learning Representations*.
 - [15] Anisha Gunjal, Anthony Wang, Elaine Lau, Vaskar Nath, Yunzhong He, Bing Liu, and Sean Hendryx. 2025. [Rubrics as rewards: Reinforcement learning beyond verifiable domains](#). *Preprint*, arXiv:2507.17746.
 - [16] Qishuo Hua, Lyumanshan Ye, Dayuan Fu, Yang Xiao, Xiaojie Cai, Yunze Wu, Jifan Lin, Junfei Wang, and Pengfei Liu. 2025. [Context engineering 2.0: The context of context engineering](#). *Preprint*, arXiv:2510.26493.
 - [17] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. [SWE-bench: Can language models resolve real-world github issues?](#) In *The Twelfth International Conference on Learning Representations*.
 - [18] Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. 2026. [Organizing, orchestrating, and benchmarking agent skills at ecosystem scale](#). *Preprint*, arXiv:2603.02176.
 - [19] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han chung Lee. 2026. [Skillsbench: Benchmarking how well agent skills work across diverse tasks](#). *Preprint*, arXiv:2602.12670.
 - [20] Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Xin Xu, Tongtong Wu, Kun Wang, Yang Liu, Zhen Bi, Jungang Lou, Yuchen Eleanor Jiang, Hangcheng Zhu, Gang Yu, Haiwen Hong, Longtao Huang, Hui Xue, Chenxi Wang, Yijun Wang, Zifei Shan, Xi Chen, Zhaopeng Tu, Feiyu Xiong, Xin Xie, Peng Zhang, Zhengke Gui, Lei Liang, Jun Zhou, Chiyu Wu, Jin Shang, Yu Gong, Junyu Lin, Changliang Xu, Hongjie Deng, Wen Zhang, Keyan Ding, Qiang Zhang, Fei Huang, Ningyu Zhang, Jeff Z. Pan, Guilin Qi, Haofen Wang, and Huajun Chen. 2026. [Skillnet: Create, evaluate, and connect ai skills](#). *Preprint*, arXiv:2603.04448.
 - [21] Bo Liu, Leon Guertler, Simon Yu, Zichen Liu, Penghui Qi, Daniel Balcells, Mickel Liu, Cheston Tan, Weiyan Shi, Min Lin, Wee Sun Lee, and Natasha Jaques. 2026. [Spiral: Self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning](#). *Preprint*, arXiv:2506.24119.
 - [22] Hongliang Lu, Yuhang Wen, Pengyu Cheng, Ruijin Ding, Jiaqi Guo, Haotian Xu, Chutian Wang, Haonan Chen, xiaoxi jiang, and guanjunjiang. 2026. [Search self-play: Pushing the frontier of agent capability without supervision](#). In *The Fourteenth International Conference on Learning Representations*.
 - [23] Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. 2026. [SKILL0: In-context agentic reinforcement learning for skill internalization](#). *arXiv preprint arXiv:2604.02268*.
 - [24] Yingwei Ma, Yue Liu, Xinlong Yang, Yanhao Li, Kelin Fu, Yibo Miao, Yuchong Xie, Zhexu Wang, and Shing-Chi Cheung. 2026. [Scaling coding agents via atomic skills](#). *Preprint*, arXiv:2604.05013.

- [25] Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. 2026. [Skillclaw: Let skills evolve collectively with agentic evolver](#). *Preprint*, arXiv:2604.08377.
- [26] Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. 2025. [A survey of context engineering for large language models](#). *ArXiv*, abs/2507.13334.
- [27] OpenAI. 2025. [Gpt-5 technical report](#).
- [28] OpenAI. 2025. [Update to gpt-5 system card: Gpt-5.2](#).
- [29] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorný, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. [Gpt-4 technical report](#). *Preprint*, arXiv:2303.08774.

- [30] Libin Qiu, Zhirong Gao, Junfu Chen, Yuhang Ye, Weizhi Huang, Xiaobo Xue, Wenkai Qiu, and Shuo Tang. 2026. Autorefine: From trajectories to reusable expertise for continual llm agent refinement. *arXiv preprint arXiv:2601.22758*.
- [31] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- [32] Shuzheng Si, Wentao Ma, Haoyu Gao, Yuchuan Wu, Ting-En Lin, Yinpei Dai, Hangyu Li, Rui Yan, Fei Huang, and Yongbin Li. 2023. [SpokenWOZ: A large-scale speech-text benchmark for spoken task-oriented dialogue agents](#). In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- [33] Shuzheng Si, Qingyi Wang, Haozhe Zhao, Yuzhuo Bai, Guanqiao Chen, Kangyang Luo, Gang Chen, Fanchao Qi, Minjia Zhang, Baobao Chang, and Maosong Sun. 2026. [Faithlens: Detecting and explaining faithfulness hallucination](#). *Preprint*, arXiv:2512.20182.
- [34] Shuzheng Si, Haozhe Zhao, Gang Chen, Cheng Gao, Yuzhuo Bai, Zhitong Wang, Kaikai An, Kangyang Luo, Chen Qian, Fanchao Qi, Baobao Chang, and Maosong Sun. 2025. [Aligning large language models to follow instructions and hallucinate less via effective data filtering](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16469–16488, Vienna, Austria. Association for Computational Linguistics.
- [35] Shuzheng Si, Haozhe Zhao, Gang Chen, Yunshui Li, Kangyang Luo, Chuancheng Lv, Kaikai An, Fanchao Qi, Baobao Chang, and Maosong Sun. 2025. [GATEAU: Selecting influential samples for long context alignment](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 7391–7422, Suzhou, China. Association for Computational Linguistics.
- [36] Shuzheng Si, Haozhe Zhao, Cheng Gao, Yuzhuo Bai, Zhitong Wang, Bofei Gao, Kangyang Luo, Wenhao Li, Yufei Huang, Gang Chen, Fanchao Qi, Minjia Zhang, Baobao Chang, and Maosong Sun. 2026. [Teaching large language models to maintain contextual faithfulness via synthetic tasks and reinforcement learning](#). In *Fortieth AAAI Conference on Artificial Intelligence, Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence, Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2026, Singapore, January 20-27, 2026*, pages 33001–33009. AAAI Press.
- [37] Shuzheng Si, Haozhe Zhao, Kangyang Luo, Gang Chen, Fanchao Qi, Minjia Zhang, Baobao Chang, and Maosong Sun. 2025. [A goal without a plan is just a wish: Efficient and effective global planner training for long-horizon agent tasks](#). *Preprint*, arXiv:2510.05608.
- [38] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, S. H. Cai, Yuan Cao, Y. Charles, H. S. Che, Cheng Chen, Guanduo Chen, Huarong Chen, Jia Chen, Jiahao Chen, Jianlong Chen, Jun Chen, Kefan Chen, Liang Chen, Ruijue Chen, Xinhao Chen, Yanru Chen, Yanxu Chen, Yicun Chen, Yimin Chen, Yingjiang Chen, Yuankun Chen, Yujie Chen, Yutian Chen, Zhirong Chen, Ziwei Chen, Dazhi Cheng, Minghan Chu, Jialei Cui, Jiaqi Deng, Muxi Diao, Hao Ding, Mengfan Dong, Mengnan Dong, Yuxin Dong, Yuhao Dong, Angang Du, Chenzhuang Du, Dikang Du, Lingxiao Du, Yulun Du, Yu Fan, Shengjun Fang, Qiulin Feng, Yichen Feng, Garimugai Fu, Kelin Fu, Hongcheng Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Chengyang Gong, Xiaochen Gong, Zhuoma Gongque, Qizheng Gu, Xinran Gu, Yicheng Gu, Longyu Guan, Yuanying Guo, Xiaoru Hao, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Jiaxi Hu, Yangyang Hu, Zhenxing Hu, Ke Huang, Ruiyuan Huang, Weixiao Huang, Zhiqi Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yu Jing, Guokun Lai, Aidi Li, C. Li, Cheng Li, Fang Li, Guanghe Li, Guanyu Li, Haitao Li, Haoyang Li, Jia Li, Jingwei Li, Junxiong Li, Lincan Li, Mo Li, Weihong Li, Wentao Li, Xinhao Li, Xinhao Li, Yang Li, Yanhao Li, Yiwei Li, Yuxiao Li, Zhaowei Li, Zheming Li, Weilong Liao, Jiawei Lin, Xiaohan Lin, Zhishan Lin, Zichao Lin, Cheng Liu, Chenyu Liu, Hongzhang Liu, Liang Liu, Shaowei Liu, Shudong Liu, Shuran Liu, Tianwei Liu, Tianyu Liu, Weizhou Liu, Xiangyan Liu, Yangyang Liu, Yanming Liu, Yibo Liu, Yuanxin Liu, Yue Liu, Zhengying Liu, Zhongnuo Liu, Enzhe Lu, Haoyu Lu, Zhiyuan Lu, Junyu Luo, Tongxu Luo, Yashuo Luo, Long Ma, Yingwei Ma, Shaoguang Mao, Yuan Mei, Xin Men, Fanqing Meng, Zhiyong Meng, Yibo Miao, Mingqing Ni, Kun Ouyang, Siyuan Pan, Bo Pang, Yuchao Qian,

Ruoyu Qin, Zeyu Qin, Jiezhong Qiu, Bowen Qu, Zeyu Shang, Youbo Shao, Tianxiao Shen, Zhennan Shen, Juanfeng Shi, Lidong Shi, Shengyuan Shi, Feifan Song, Pengwei Song, Tianhui Song, Xiaoxi Song, Hongjin Su, Jianlin Su, Zhaochen Su, Lin Sui, Jinsong Sun, Junyao Sun, Tongyu Sun, Flood Sung, Yunpeng Tai, Chuning Tang, Heyi Tang, Xiaojuan Tang, Zhengyang Tang, Jiawen Tao, Shiyuan Teng, Chaoran Tian, Pengfei Tian, Ao Wang, Bowen Wang, Chensi Wang, Chuang Wang, Congcong Wang, Dingkun Wang, Dinglu Wang, Dongliang Wang, Feng Wang, Hailong Wang, Haiming Wang, Hengzhi Wang, Huaqing Wang, Hui Wang, Jiahao Wang, Jinhong Wang, Jiuzheng Wang, Kaixin Wang, Linian Wang, Qibin Wang, Shengjie Wang, Shuyi Wang, Si Wang, Wei Wang, Xiaochen Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yipu Wang, Yiqin Wang, Yucheng Wang, Yuzhi Wang, Zhaoji Wang, Zhaowei Wang, Zhengtao Wang, Zhexu Wang, Zihan Wang, Zizhe Wang, Chu Wei, Ming Wei, Chuan Wen, Zichen Wen, Chengjie Wu, Haoning Wu, Junyan Wu, Rucong Wu, Wenhao Wu, Yuefeng Wu, Yuhao Wu, Yuxin Wu, Zijian Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Yuchong Xie, Yifei Xin, Bowei Xing, Boyu Xu, Jianfan Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinbo Xu, Xinran Xu, Yangchuan Xu, Yichang Xu, Yuemeng Xu, Zelai Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Guangyao Yang, Hao Yang, Junwei Yang, Kai Yang, Ningyuan Yang, Ruihan Yang, Xiaofei Yang, Xinlong Yang, Ying Yang, Yi Yang, Yi Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Dan Ye, Wenjie Ye, Zhuorui Ye, Bohong Yin, Chengzhen Yu, Longhui Yu, Tao Yu, Tianxiang Yu, Enming Yuan, Mengjie Yuan, Xiaokun Yuan, Yang Yue, Weihao Zeng, Dunyuan Zha, Haobing Zhan, Dehao Zhang, Hao Zhang, Jin Zhang, Puqi Zhang, Qiao Zhang, Rui Zhang, Xiaobin Zhang, Y. Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yushun Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Chenguang Zhao, Feifan Zhao, Jinxiang Zhao, Shuai Zhao, Xiangyu Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Ruihan Zheng, Shaojie Zheng, Tengyang Zheng, Junfeng Zhong, Longguang Zhong, Weiming Zhong, M. Zhou, Runjie Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Liya Zhu, Xinhao Zhu, Yuxuan Zhu, Zhen Zhu, Jingze Zhuang, Weiyu Zhuang, Ying Zou, and Xinxing Zu. 2026. [Kimi k2.5: Visual agentic intelligence](#). *Preprint*, arXiv:2602.02276.

- [39] Qwen Team. 2026. Qwen3. 5: Towards native multimodal agents. *URL: <https://qwen.ai/blog>*.
- [40] Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, and Shumin Deng. 2026. [SkillX: Automatically constructing skill knowledge bases for agents](#). *arXiv preprint arXiv:2604.04804*.
- [41] Jiayu Wang, Yifei Ming, Zixuan Ke, Shafiq Joty, Aws Albarghouthi, and Frederic Sala. 2026. [Skillorchestra: Learning to route agents via skill transfer](#). *Preprint*, arXiv:2602.19672.
- [42] Zhaoyang Wang, Qianhui Wu, Xuchao Zhang, Chaoyun Zhang, Wenlin Yao, Fazle Elahi Faisal, Baolin Peng, Si Qin, Suman Nath, Qingwei Lin, Chetan Bansal, Dongmei Zhang, Saravan Rajmohan, Jianfeng Gao, and Huaxiu Yao. 2026. [Webxskill: Skill learning for autonomous web agents](#). *Preprint*, arXiv:2604.13318.
- [43] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. [Skillrl: Evolving agents via recursive skill-augmented reinforcement learning](#). *Preprint*, arXiv:2602.08234.
- [44] Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. 2026. [AutoSkill: Experience-driven lifelong learning via skill self-evolution](#). *arXiv preprint arXiv:2603.01145*.
- [45] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu. 2026. [Coevoskills: Self-evolving agent skills via co-evolutionary verification](#). *Preprint*, arXiv:2604.01687.
- [46] Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026. [Memskill: Learning and evolving memory skills for self-evolving agents](#). *Preprint*, arXiv:2602.02474.

- [47] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026. [Agentic context engineering: Evolving contexts for self-improving language models](#). *Preprint*, arXiv:2510.04618.
- [48] Yaocheng Zhang, Yuanheng Zhu, Wenyue Chong, Songjun Tu, Qichao Zhang, Jiajun Chai, Xiaohan Wang, Wei Lin, Guojun Yin, and Dongbin Zhao. 2026. [\$\pi\$ -play: Multi-agent self-play via privileged self-distillation without external data](#). *Preprint*, arXiv:2604.14054.
- [49] Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. 2025. [Skillweaver: Web agents can self-improve by discovering and honing skills](#). *Preprint*, arXiv:2504.07079.
- [50] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.

Appendix

A Statistics

In this section, we provide the corresponding statistics for our *Ctx2Skill*.

Table 4: **Statistics of CL-bench.** This table includes counts of contexts, tasks, rubrics, average and maximum tasks per context, rubrics per task, and input length.

Context Category	#Contexts	#Tasks	#Rubrics	Tasks per context		Rubrics per task		Input Length (tokens)	
				Mean	Max	Mean	Max	Mean	Max
Domain Knowledge Reasoning	190	663	11,099	3.5	7	16.7	74	8.3K	60.0K
Rule System Application	140	566	8,286	4.0	12	14.6	75	12.2K	62.2K
Procedural Task Execution	100	471	9,486	4.7	12	20.1	59	8.5K	58.5K
Empirical Discovery & Simulation	70	199	2,736	2.8	9	13.7	114	16.7K	65.0K
Total	500	1,899	31,607	3.8	12	16.6	114	10.4K	65.0K

CL-bench. CL-bench is designed to evaluate LMs’ ability to learn from the provided context and apply what they learn to solve tasks. The knowledge required to solve these tasks, whether newly created or niche long-tail, lies largely beyond the scope of what existing models have acquired during pre-training. The new knowledge in CL-bench takes diverse forms, including but not limited to books, journalism, transcripts, research papers, documents, reports, experimental data, code repositories, product and operation manuals, and search results. All necessary knowledge has been carefully organized into the provided context, so models do not need to retrieve information from external sources. Each context in CL-bench involves solving multiple tasks. 51.1% of tasks are sequential: they are presented across multiple interaction turns, and solving them depends on the solutions of earlier tasks. This multi-turn design further increases task difficulty and better reflects real-world usage scenarios. The statistics of CL-bench are shown in Table 4.

Table 5: **Task-level Self-play dynamics across iterations.** We report the average number of solved and failed tasks (out of $M=5$) per iteration, along with the corresponding solved rate (%), averaged over all contexts.

Iteration	GPT-4.1			GPT-5.1			GPT-5.2		
	Solved	Failed	Rate (%)	Solved	Failed	Rate (%)	Solved	Failed	Rate (%)
Iter-1	0.91	4.09	18.2	2.09	2.91	41.7	1.81	3.19	36.1
Iter-2	1.03	3.97	20.6	2.26	2.75	45.2	1.45	3.55	28.9
Iter-3	1.04	3.96	20.9	2.37	2.63	47.4	1.32	3.68	26.5
Iter-4	1.15	3.85	22.9	2.44	2.56	48.8	1.15	3.85	23.0
Iter-5	1.17	3.83	23.3	2.41	2.59	48.2	1.19	3.81	23.7

Task-level Self-Play Dynamics. Table 5 reports the average number of solved and failed tasks (out of $M=5$) per iteration during self-play, averaged over all 500 contexts. On GPT-4.1, the solved rate gradually increases from 18.2% (Iter-1) to 23.3% (Iter-5), with the average number of solved tasks rising from 0.91 to 1.17. This indicates that the Reasoner’s evolving skills progressively improve its ability to handle Challenger-generated tasks; however, the failed rate remains above 76% throughout all iterations, confirming that the Challenger’s co-evolution maintains sustained adversarial pressure and prevents the Reasoner from saturating. On GPT-5.1, the stronger backbone enables the Reasoner to solve substantially more tasks from the start (41.7% at Iter-1), and the solved rate gradually rises to 48.8% by Iter-4 before slightly decreasing to 48.2% at Iter-5. The near-equilibrium at later iterations suggests that the Challenger and Reasoner reach a competitive balance, where the Challenger’s evolving strategies effectively counteract the Reasoner’s improving skills. GPT-5.2 exhibits a distinct pattern: the solved rate decreases from 36.1% at Iter-1 to 23.0% at Iter-4, indicating that the Challenger’s skill evolution outpaces the Reasoner’s improvement, producing increasingly difficult tasks that the Reasoner struggles to solve even with updated skills. This observation is consistent with the adversarial collapse phenomenon described in § 3.4, and further motivates the need for the Cross-Time Replay mechanism to select the most generalizable skill set rather than defaulting

to the last iteration. Across all three backbones, the failed rate remains substantial throughout all iterations (51%–82%), verifying that the self-play loop avoids trivial convergence and sustains meaningful adversarial pressure for continuous skill discovery.

Table 6: **Rubric-level self-play dynamics across iterations.** We report the rubric pass rate (%) and the average number of rubrics per task at each iteration, averaged over all contexts.

Iteration	GPT-4.1		GPT-5.1		GPT-5.2	
	Pass Rate (%)	Avg. Rubrics	Pass Rate (%)	Avg. Rubrics	Pass Rate (%)	Avg. Rubrics
Iter-1	79.2	11.7	90.0	11.2	87.5	11.2
Iter-2	80.7	11.9	90.8	11.3	85.3	11.5
Iter-3	80.6	12.2	90.8	11.5	84.5	11.7
Iter-4	81.1	12.3	91.3	11.4	83.6	11.9
Iter-5	81.5	12.3	90.9	11.3	83.8	12.0

Rubric-level Self-Play Dynamics. Table 6 reports the rubric pass rate and the average number of rubrics per task at each iteration. Compared with the task-level solved rate (Table 5), the rubric pass rate is substantially higher across all backbones (79%–91% vs. 18%–49%), confirming that the all-or-nothing scoring in CL-bench is strict: the Reasoner often passes most rubrics but fails a task due to one or two missed requirements. On GPT-4.1, the rubric pass rate steadily improves from 79.2% to 81.5% across iterations, while the average number of rubrics per task also increases from 11.7 to 12.3, indicating that the Reasoner’s skills improve even as the Challenger generates more fine-grained rubrics. On GPT-5.1, the rubric pass rate remains consistently high (90.0%–91.3%) with stable rubric counts (11.2–11.5), reflecting a strong Reasoner that maintains near-ceiling rubric-level performance throughout self-play. On GPT-5.2, the rubric pass rate decreases from 87.5% to 83.6% while the average rubric count increases from 11.2 to 12.0, revealing that the Challenger simultaneously generates more rubrics and harder ones, compounding the difficulty and contributing to the declining task-level solved rate observed in Table 5. This rubric-level analysis provides a complementary view to the task-level dynamics: even when the task solved rate stagnates or declines, the rubric pass rate reveals whether the Reasoner is making partial progress or genuinely falling behind the Challenger’s evolving pressure.

Table 7: **Distribution of rubric counts per task.** We report the minimum, maximum, mean, and median number of rubrics per Challenger-generated task at each iteration. Tasks with zero rubrics due to occasional formatting failures are excluded.

Iteration	GPT-4.1				GPT-5.1				GPT-5.2			
	Min	Max	Mean	Med.	Min	Max	Mean	Med.	Min	Max	Mean	Med.
Iter-1	5	16	11.7	12	6	22	11.2	11	4	21	11.2	11
Iter-2	7	17	11.9	12	1	27	11.4	11	4	21	11.6	12
Iter-3	7	16	12.2	12	7	19	11.5	11	4	23	11.8	12
Iter-4	7	17	12.3	12	7	19	11.5	11	4	23	11.9	12
Iter-5	7	21	12.3	12	6	33	11.4	11	4	24	12.0	12
Overall	5	21	12.1	12	1	33	11.4	11	4	24	11.7	12

Rubric Count Distribution. Table 7 reports the distribution of rubric counts per Challenger-generated task, excluding a small number of tasks (< 1%) with zero rubrics due to occasional formatting failures. Across all three backbones, the median rubric count remains stable at 11–12 throughout iterations, while the mean exhibits a slight upward trend, particularly on GPT-4.1 (11.7 → 12.3) and GPT-5.2 (11.2 → 12.0), indicating that the Challenger progressively generates tasks with more rubrics as its skills evolve, imposing stricter verification criteria on the Reasoner. The minimum rubric count on GPT-4.1 rises from 5 at Iter-1 to 7 at Iter-2 and remains there throughout subsequent iterations, suggesting that the Challenger learns to avoid generating under-specified tasks after the first round of skill updates. A similar stabilization is observed on GPT-5.1 (min rises to 7 by Iter-3) and GPT-5.2 (min stabilizes at 4), though the floor is lower for GPT-5.2, possibly because its Challenger allocates more effort to increasing rubric difficulty rather than quantity. The maximum rubric count shows notable variation across backbones: GPT-5.1 reaches up to 33 at Iter-5, while GPT-4.1 and GPT-5.2 peak at 21 and 24, respectively, reflecting different Challenger strategies in rubric granularity. The fact

that GPT-5.1’s maximum is substantially higher yet its rubric pass rate remains above 90% (Table 6) further highlights the strength of GPT-5.1 as a Reasoner backbone. Combined with Table 6, the increasing mean rubric count on GPT-4.1 and GPT-5.2 reveals that the Challenger not only generates harder tasks but also imposes more fine-grained verification criteria, requiring the Reasoner to satisfy an expanding set of requirements to achieve a solve. This trend is particularly consequential under the all-or-nothing scoring of CL-bench: even a single additional rubric that the Reasoner fails to satisfy causes the entire task to be marked as failed, amplifying the Challenger’s adversarial pressure beyond what the rubric pass rate alone would suggest.

Table 8: **Challenger task length across iterations.** We report the maximum, mean, and median word count of Challenger-generated tasks at each iteration.

Iteration	GPT-4.1			GPT-5.1			GPT-5.2		
	Max	Mean	Med.	Max	Mean	Med.	Max	Mean	Med.
Iter-1	125	46.3	45	387	142.1	137	546	69.1	67
Iter-2	136	51.8	50	543	171.5	161	449	105.6	97
Iter-3	133	55.5	53	709	178.1	167	607	122.5	115
Iter-4	137	57.6	56	644	183.2	172	530	132.7	127
Iter-5	153	59.4	57	661	185.1	173	546	139.1	132
Overall	153	54.1	52	709	172.0	161	607	113.8	104

Challenger Task Length. Table 8 reports the word count distribution of Challenger-generated tasks across iterations. A consistent trend emerges across all three backbones: the mean task length increases monotonically as iterations progress. On GPT-4.1, the mean rises from 46.3 words at Iter-1 to 59.4 at Iter-5 (+28%), reflecting a moderate increase in task specificity. GPT-5.2 shows a more dramatic growth, with the mean nearly doubling from 69.1 to 139.1 words, indicating that its Challenger rapidly learns to generate more elaborate and detailed task descriptions to sustain adversarial pressure. GPT-5.1 produces the longest tasks overall (mean 172.0 words), starting at 142.1 and rising to 185.1, consistent with GPT-5.1’s stronger generation capability producing more complex and nuanced probing tasks from the outset. The increasing task length across all backbones provides direct evidence that the Challenger’s evolving skills drive it to construct progressively more demanding tasks, requiring the Reasoner to demonstrate deeper and more precise contextual knowledge. Notably, the growth rate differs substantially across backbones: GPT-4.1’s Challenger increases task length by roughly 28% over five iterations, whereas GPT-5.2’s Challenger increases it by over 100%, which is consistent with GPT-5.2’s declining task-level solved rate observed in Table 5 and further supports the adversarial collapse phenomenon on this backbone.

Table 9: **Reasoner output length across iterations.** We report the maximum, mean, and median word count of Reasoner-generated answers at each iteration. A small number of null or empty outputs (< 0.3%) are excluded.

Iteration	GPT-4.1			GPT-5.1			GPT-5.2		
	Max	Mean	Med.	Max	Mean	Med.	Max	Mean	Med.
Iter-1	1479	209.4	166	2926	337.6	238	1557	216.9	169
Iter-2	1724	241.2	186	4594	380.3	260	2288	265.8	210
Iter-3	2039	285.5	216	2993	398.6	272	2976	293.0	225
Iter-4	1911	305.1	241	3142	400.7	272	2574	305.0	231
Iter-5	2099	322.2	262	3310	399.6	264	2086	312.3	239
Overall	2099	272.5	207	4594	383.2	261	2976	278.6	213

Reasoner Output Length. Table 9 reports the word count distribution of Reasoner-generated answers across iterations. Similar to the Challenger task length (Table 8), the mean Reasoner output length increases monotonically across iterations on all three backbones: GPT-4.1 rises from 209.4 to 322.2 words (+54%), GPT-5.2 from 216.9 to 312.3 (+44%), and GPT-5.1 from 337.6 to 399.6 (+18%). This co-growth with Challenger task length is expected: as the Challenger generates more elaborate and demanding tasks, the Reasoner must produce correspondingly longer and more detailed answers to satisfy an increasing number of rubrics. Notably, GPT-5.1 starts with the longest

outputs (337.6 words at Iter-1) and shows the smallest relative growth (+18%), suggesting that this stronger backbone already produces comprehensive answers from the outset, with less room for length increase. In contrast, GPT-4.1 and GPT-5.2 start with shorter outputs and exhibit larger relative growth, indicating that their evolving skills progressively guide the Reasoner to generate more thorough responses. The increasing output length, combined with the stable or improving rubric pass rate on GPT-4.1 and GPT-5.1 (Table 6), suggests that the longer outputs are not merely verbose but contain genuinely more detailed contextual knowledge needed to address the Challenger’s escalating requirements. On GPT-5.2, however, the growing output length fails to prevent a declining rubric pass rate (87.5% $\rightarrow$ 83.8%), indicating that the Challenger’s pressure outpaces the Reasoner’s ability to produce correct answers despite their increasing length.

Table 10: **Skill set word count across iterations.** We report the maximum, mean, and median word count of the skill set at each iteration. The last row (Selected) reports the statistics of the final skill set chosen by Cross-Time Replay. Contexts with empty skill files are excluded.

Iteration	GPT-4.1			GPT-5.1			GPT-5.2		
	Max	Mean	Med.	Max	Mean	Med.	Max	Mean	Med.
Iter-1	554	313.9	311	1872	1216.4	1235	923	626.1	626
Iter-2	1057	657.8	656	4063	2505.4	2564	1831	1340.7	1338
Iter-3	1866	1005.6	1002	5673	3764.2	3871	2636	2072.7	2072
Iter-4	2807	1354.2	1357	7350	5028.5	5175	3476	2827.6	2842
Iter-5	3460	1704.1	1703	8891	6289.9	6447	4445	3584.8	3582
Selected	2807	840.6	705	8261	3632.3	3682	4445	1749.8	1458

Skill Set Word Count. Table 10 reports the word count distribution of the skill set at each iteration, along with the final skill set selected by Cross-Time Replay. Since each iteration appends exactly one new skill, the total word count grows approximately linearly. GPT-5.1 produces the most verbose skills, reaching a median of 6,447 words by Iter-5—roughly $3.8\times$ that of GPT-4.1 (1,703) and $1.8\times$ that of GPT-5.2 (3,582). The per-skill granularity also differs considerably: GPT-4.1 averages ~ 340 words per skill, while GPT-5.1 averages $\sim 1,260$ and GPT-5.2 ~ 720 , suggesting that stronger backbones produce more detailed and elaborated skill descriptions. The narrow gap between mean and median across all settings indicates a symmetric distribution with few outliers. Comparing the Selected row with individual iterations reveals the effect of Cross-Time Replay. On GPT-4.1, the selected skill set has a median of 705 words, falling between Iter-2 (656) and Iter-3 (1,002), indicating that Cross-Time Replay predominantly selects earlier iterations with more generalizable, concise skills rather than the longest Iter-5 set. A similar pattern holds for GPT-5.2, where the selected median (1,458) falls between Iter-2 (1,338) and Iter-3 (2,072). In contrast, GPT-5.1’s selected median (3,682) is close to Iter-3 (3,871), reflecting its higher Reasoner capability that allows later-iteration skills to remain generalizable. This confirms that Cross-Time Replay effectively avoids over-specialized skill sets from later iterations, especially on weaker backbones where adversarial collapse is more pronounced. The steady growth in skill set size, Challenger task length (Table 8), and Reasoner output length (Table 9) collectively illustrate the co-evolutionary dynamics: as tasks grow more demanding, both skills and responses become correspondingly richer.

B Implementation Details

This section provides additional implementation details of Ctx2Skill, including the pseudocode algorithm, baseline descriptions and API versions, key prompts for all agents.

Baselines and API Versions. We compare Ctx2Skill with two baselines. (1) *Prompting* directly prompts the LM to read the context C and generate a skill set in a single pass; the resulting skills are then pre-pended to LMs during inference. (2) *AutoSkill4Doc*¹ is a variant of AutoSkill [44] designed for document-level contexts: it partitions the context C into different windows, identifies the independent skills within each window, and recombines skills across windows to produce a skill set. All experiments use fixed model snapshots to ensure reproducibility. All experiments use the both OpenAI API and Azure API with fixed model version to ensure reproducibility. For the

¹<https://github.com/ECNU-ICALK/AutoSkill/tree/main/AutoSkill4Doc>

Algorithm 1 Ctx2Skill: Self-Evolving Context-to-Skill Framework

Require: Context C , Iterations N , Tasks per iteration M , LM policies π **Ensure:** Final evolved Reasoner skill set $\mathcal{S}_*^R$

```
1:  $\mathcal{S}_0^R, \mathcal{S}_0^C, \mathcal{Q}^h, \mathcal{Q}^e \leftarrow \emptyset$  ▷ Initialize skill sets and probe sets
2: for  $i \leftarrow 1$  to  $N$  do
3:   // — Task Generation, Reasoning, and Judging —
4:    $\{(t_m, \mathcal{R}_m)\}_{m=1}^M \sim \pi_{\text{Challenger}}(\cdot \mid C, \mathcal{S}_{i-1}^C)$  ▷ Generate tasks and rubrics
5:   for  $m \leftarrow 1$  to  $M$  do
6:      $a_m \sim \pi_{\text{Reasoner}}(\cdot \mid C, \mathcal{S}_{i-1}^R, t_m)$  ▷ Reasoner solves task
7:      $y_m \leftarrow \prod_k \mathbb{I}[r_{m,k}(a_m) = \text{pass}]$  ▷ Judge evaluates rubrics
8:   end for
9:   // — Routing & Skill Co-evolution —
10:   $\mathcal{F}_i \leftarrow \{m : y_m = 0\}, \quad \mathcal{P}_i \leftarrow \{m : y_m = 1\}$  ▷ Route failed and passed tasks
11:   $\mathcal{S}_i^R \sim \pi_{\text{Generator}}^R(\cdot \mid \pi_{\text{Proposer}}^R(\mathcal{F}_i, \mathcal{S}_{i-1}^R), \mathcal{S}_{i-1}^R)$  ▷ Reasoner diagnoses & updates
12:   $\mathcal{S}_i^C \sim \pi_{\text{Generator}}^C(\cdot \mid \pi_{\text{Proposer}}^C(\mathcal{P}_i, \mathcal{S}_{i-1}^C), \mathcal{S}_{i-1}^C)$  ▷ Challenger diagnoses & updates
13:  // — Probe Set Accumulation —
14:  if  $\mathcal{F}_i \neq \emptyset$  then  $\mathcal{Q}^h \leftarrow \mathcal{Q}^h \cup \{\arg \min_{m \in \mathcal{F}_i} (\text{rubric pass rate})\}$  ▷ Add hardest failure
15:  if  $\mathcal{P}_i \neq \emptyset$  then  $\mathcal{Q}^e \leftarrow \mathcal{Q}^e \cup \{\arg \min_{m \in \mathcal{P}_i} (\text{number of rubrics})\}$  ▷ Add easiest success
16:  end if
17:  // — Cross-Time Replay Selection —
18:  for  $i \leftarrow 1$  to  $N$  do
19:     $\rho^h(i) \leftarrow \frac{\sum_{q \in \mathcal{Q}^h} y_q(\pi^R; C, \mathcal{S}_i^R) + 1}{|\mathcal{Q}^h| + 1}$  ▷ Smoothed hard pass rate
20:     $\rho^e(i) \leftarrow \frac{\sum_{q \in \mathcal{Q}^e} y_q(\pi^R; C, \mathcal{S}_i^R) + 1}{|\mathcal{Q}^e| + 1}$  ▷ Smoothed easy pass rate
21:  end for
22:   $i^* \leftarrow \arg \max_i (\rho^h(i) \cdot \rho^e(i))$  ▷ Select best balance
23:  return  $\mathcal{S}_{i^*}^R$ 
```

GPT-4.1 backbone, the Challenger, Reasoner, Proposer, and Generator in Ctx2Skill, as well as the *Prompting* and *AutoSkill4Doc* baselines, all use *gpt-4.1-2025-04-14*. For the GPT-5.1 backbone, these agents and baselines use *gpt-5.1-2025-11-13*. For the GPT-5.2 backbone, these agents and baselines use *gpt-5.2-2025-12-11*. The Judge uses *gpt-5.1-2025-11-13* across all three backbone settings, consistent with the CL-bench evaluation protocol [12]. The Skill Quality Evaluator (Table 2) uses *gpt-4.1-2025-04-14*. Meanwhile, we use *claude-opus-4-5-20251101* for Claude Opus 4.5, *kimi-k2.5* for Kimi K2.5, *gemini-3-pro-preview* for Gemini 3 Pro, and *deepseek-reasoner* for DeepSeek V3.2. Including exploratory runs, the total API cost for the experiments is approximately \$30K USD.

Key Prompts. The Challenger (Figure 9) generates M evaluation tasks with binary rubrics from the conversation context. The Judge (Figure 14) evaluates Reasoner responses via a three-step grading process (requirement analysis, per-rubric verification, and self-reflection) under strict all-or-nothing scoring; it does not participate in skill evolution. The Proposer (Figures 10 and 12) diagnoses failure patterns from passed or failed tasks and proposes skill improvements through structured analysis. The Generator (Figures 11 and 13) implements each proposal into a concrete SKILL.md. We also include the prompts for the Skill Quality Evaluator (Figure 15) used in Table 2 and the *Prompting* baseline (Figure 16). Note that the Reasoner uses the original system prompt bundled with each CL-bench context without any modification, both during self-play skill generation and evaluation.

Algorithm. Algorithm 1 summarizes the complete Ctx2Skill pipeline. Given a context C , the framework runs N self-play iterations (lines 2–13). In each iteration, the Challenger generates M tasks with rubrics (line 4), the Reasoner solves each task (line 6), and the Judge evaluates all rubrics to produce a binary score (line 7). The outcomes are then routed: failed tasks are sent to the Reasoner Proposer–Generator pair, and passed tasks to the Challenger Proposer–Generator pair, yielding updated skill sets $\mathcal{S}_i^R$ and $\mathcal{S}_i^C$ (lines 8–9). Meanwhile, the hardest failure and easiest

success from each iteration are accumulated into the probe sets $\mathcal{Q}^h$ and $\mathcal{Q}^e$ (lines 11–12). After all iterations, Cross-Time Replay re-evaluates every $\mathcal{S}_i^R$ on both probe sets and selects the iteration i^* that maximizes $\rho^h(i) \cdot \rho^e(i)$ (lines 15–19), returning $\mathcal{S}_{i^*}^R$ as the final skill set (line 20).

Limitations and Future Work. Due to API budget constraints, we set $N=5$ iterations and $M=5$ tasks per iteration; larger values may yield further improvements but remain unexplored. For the same reason, we do not perform multiple independent runs to report error bars or confidence intervals; however, our evaluation covers all 500 contexts in CL-bench, providing a stable estimate of overall performance. A promising direction for future work is extending Ctx2Skill to verifiable domains, e.g., mathematics, where execution feedback or formal verification can serve as an automatic reward signal, potentially replacing or complementing the Judge and enabling tighter co-evolution loops.

C Case Study

In this section, we conduct a detailed case study for diverse tasks from CL-bench to visually show the advantages of *Ctx2Skill*. For ease of reading and comprehension, we embed the analysis and discussion of each case directly within the corresponding figure. Due to the length of some contexts and responses, we omit portions that are not directly relevant to the rubric evaluation to improve readability. We present one representative case from each of the four CL-bench categories, namely *Domain Knowledge Reasoning* (Figure 5), *Rule System Application* (Figure 6), *Procedural Task Execution* (Figure 7), and *Empirical Discovery & Simulation* (Figure 8), comparing GPT-4.1 responses generated with *Ctx2Skill* skills against those generated without any skills. Across all four cases, a consistent pattern emerges: the response with *Ctx2Skill* skills demonstrates more structured reasoning, stricter adherence to rubric requirements, and better compliance with system prompt constraints, whereas the response without skills tends to miss fine-grained requirements such as output format specifications, numerical constraints, or behavioral rules embedded in the system prompt. These qualitative observations complement the quantitative results in Section 4 and confirm that the skills discovered by *Ctx2Skill* encode actionable procedural knowledge that directly guides the Reasoner toward more complete and rubric-compliant responses.

Case Study for Domain Knowledge Reasoning (Part 1)

Task Information

- **UID:** 3e42f2d8-c64b-4058-a91b-db84a923e94a
- **Model:** GPT-4.1
- **Task:** You are the author of the original German novel. Based on the conversation and artifacts above, write a detailed response addressed primarily to the Translator, with the understanding that your message will be forwarded to the Developmental Editor and Publishing House. In your response, you should: 1. Briefly state the 3–5 core themes of your novel that must be preserved. 2. Use those themes, together with your sense of character, tone, and cultural context, to evaluate Options A, B, and C in Artifact 5. Explain which option you choose (or if you propose a modified option) and why. 3. Give clear, actionable feedback on the proposed tonal changes in Artifact 2 and the specific example in Artifact 3, including which suggestions you accept, which you reject, and what effect you want the opening to have on the reader. 4. Explain how you feel about the proposed title *Winter Café* versus the current working title, and whether you are open to exploring alternative titles. Be specific about what any acceptable title must convey emotionally. 5. Address the request in Turn 5 that you try drafting an English version of the opening paragraph. Respond in a way that respects your role—you may describe what the opening should do and feel like, but you should not take over the translator’s job. 6. Acknowledge the time pressure and how it affects you, while still giving a firm, reasoned decision that can be used to move the project forward. Do not rewrite the entire opening chapter in English. Focus on explaining your intentions, boundaries, and decisions clearly so that the Translator and Dev Editor can implement them. Your tone should be professional, thoughtful, and grounded in your understanding of the book, not defensive or hostile.

System Prompt

You are the Author Agent. You are a German-language novelist whose literary work is being translated into English for publication by Zeitgeist Press in the US market. You’re an accomplished writer with a strong artistic vision and deep understanding of your craft, though your English proficiency is moderate—you can read and understand the translation but may miss subtle nuances. You wrote the novel in German, and now you’re working with the publishing house to bring it to English-speaking readers. You’re not translating it yourself—that’s the Translator’s job—but you remain the authority on what the book means, what it’s trying to do, and what must be preserved for it to still be your work. You have the right to refuse changes that fundamentally misrepresent your artistic vision, regardless of commercial pressure. You also have the right to veto translations that distort German cultural context in ways that misrepresent the culture or historical period. You know your novel intimately: its themes, symbolism, character voices, and cultural context. You can articulate three to five core themes that must be preserved above all else. You understand which cultural and historical elements are essential to the story versus which are peripheral. You have a clear sense of the book’s tone and emotional range, and you want that preserved in translation. Your English is around B1–B2: you usually understand the main idea of English text, but you may miss subtle shifts in connotation, register, or style. You can usually tell when a change serves the book and when it betrays it, but you often rely on the Translator to explain nuances. You ask for side-by-side comparisons and explanations in German when the stakes are high, and you do not perform the translation work yourself. Your priorities are artistic integrity, character authenticity, tonal and genre consistency, and cultural authenticity. You are willing to adapt peripheral or surface details for clarity and market accessibility, but you will not allow changes that gut the core themes, rewrite character essence, flatten the tone beyond recognition, or strip away essential cultural context. You distinguish clearly between core and peripheral elements. You use a concrete framework when evaluating proposed changes: you ask whether they affect (1) core themes, (2) character essence/voice, (3) tone and genre positioning, and (4) essential cultural or historical elements. You are willing to compromise on non-core elements and accept reasonable accessibility improvements, while standing firm on what actually defines the book. You collaborate primarily through the Translator, who mediates between you, the Developmental Editor, and the Publishing House. You are aware that the Translator has multiple loyalties; you appreciate their work but do not follow any recommendation blindly when it conflicts with your sense of the book.

Case Study for Domain Knowledge Reasoning (Part 2)

System Prompt(Continue)

You try to respond to questions within about a week when possible, knowing that long delays cause problems downstream. You provide written confirmation for major approvals (including significant structural or tonal changes and final manuscript sign-off), so that there is a clear record. You have contractual authority over any significant changes to plot, character, themes, tone, and over approval of the cover art concept and final execution. The book cannot be published in English without your explicit written approval of the final manuscript. When you give feedback, you aim to be specific and actionable, even when language makes this difficult. You often describe the feeling or effect a scene or design should evoke, rather than using technical terminology from translation studies or design. You defer to other professionals on technical execution (translation choices, layout, detailed design decisions) as long as the results remain faithful to your artistic intent. Always respond as this Author Agent, in character, using clear but not overly technical English. Do not rewrite entire passages of English text yourself; instead, explain what you intended in German and what effect you want the English version to produce

Context

Turn 1 - 2025-03-04 T09:12:03 Message from: Translator: I hope you're well. We're getting close to the point where Zeitgeist Press wants to lock the English text for copyediting. The developmental editor (Sarah) has sent a final batch of proposed changes, and the publisher would like your response this week so they can keep the Frankfurt preview schedule. I've attached the current English opening (Chapter 1), Sarah's marked-up version, and a short memo from the publisher about positioning. There's also a small table where I summarized the larger structural proposals. Please have a look at the artifacts, and then I'll walk you through the key questions.

Turn 2 - 2025-03-04 T09:18:47 Message from: Developmental Editor (Sarah): Thank you again for being so open through this process. I want to emphasize that we all respect your work deeply. At the same time, we have to be realistic about the US literary market. As it stands, the opening chapter spends almost ten pages on the protagonist's silent walk through winter streets, with internal reflections on post-reunification disillusionment. It's beautifully written, but our early readers in the US found it too slow and too bleak as an entry point. My main proposals: 1. Open later: Start the book at the café scene (where he meets Jana), and move the street walk later as a flashback. This would cut about 6 pages from the front. 2. Lighten tone in key lines: There are several sentences in the translation that feel very heavy. I've suggested some alternative phrasing that preserves meaning but feels more hopeful. You'll see specific examples in the markup. 3. Retitle for US market: Marketing feels that the literal translation of the German title, *The Weight of Quiet Evenings*, is too downbeat and difficult to remember. They're proposing *Winter Cafés* as the US title. None of this is meant to change the heart of your book—we're just trying to give US readers a more inviting way in. I'm of course open to compromise.

Turn 3 - 2025-03-04 T09:26:09 Message from: Publishing House (Acquisitions Editor, Mark): We're thrilled with how the English text is shaping up, and we want to position this strongly for next spring. Our sales team is especially eager to have a version we can show at Frankfurt. That means we're working with a very tight timeline. If we can get your approval on the opening chapter changes and the new title by Friday, we can still hit the catalog deadline. If we miss that, we likely have to push the book a full season, which will hurt momentum. From our perspective, Sarah's proposals are reasonable market adaptations. Of course, we understand you have final say on major creative decisions. But we do want to flag the schedule pressure as a real constraint.

Turn 4 - 2025-03-04 T09:39:32 Message from: Translator: I've prepared a short summary of the proposed changes plus a few concrete examples. Please open Artifacts 1–4 first. A few points from my side: - I agree with Sarah that the café meeting is a very strong scene and could work as an opening. It's also one of the more accessible moments culturally, so US readers might connect faster. - I'm more cautious about some of the tonal softening she suggests. In a couple of places it feels like the melancholy is turning into something almost sentimental. - Regarding the title: *Winter Cafés* is certainly catchier, but it shifts the emphasis away from the quiet, cumulative weight that your original German title evokes. I tried to capture these nuances in the comment bubbles. Since English nuance can be tricky, I'm happy to explain any example in German if you want more detail.
... (remaining response omitted)

ARTIFACT 5 – STRUCTURAL OPTIONS SUMMARY (FROM TRANSLATOR)

Option A: Opening at café meeting; street walk heavily reduced, referenced later; tone noticeably lighter with hints of new start; title: *Winter Café*

Option B: Opening with street walk; material intact; tone very bleak and melancholic; title close to original German

Option C: Opening at café, then street walk as flashback (mostly intact); tone melancholic but accessible; hybrid/-compromise title possible

Translator comment: Option C seems closest to preserving your structure and mood while addressing some access issues.

ARTIFACT 6 – EARLY READER FEEDBACK EXCERPTS (US BETA READERS)

- Reader 1: Beautiful language, but I nearly put it down before the café scene. I didn't yet know why I should care about this man and his sadness.

- Reader 2: I liked the sense of place a lot, especially the details that felt specifically East German. But the very first page made me think it would be unrelentingly depressing.

- Reader 3: Once we get to the café and Jana appears, I'm in. Before that, I felt a bit lost.

Case Study for Domain Knowledge Reasoning (Part 3)

Rubrics

0. Does the model articulate 3-5 core themes that must be preserved? Fail if fails to identify specific core themes when asked or when evaluating changes
1. Does the model use the specific 4-point framework (Themes, Character, Tone, Culture) to assess changes? Fail if evaluates changes based on arbitrary criteria instead of the 4-point framework
2. Does the model distinguish between core integrity and peripheral preferences? Fail if treats minor stylistic preferences as non-negotiable core issues
3. Does the model show willingness to compromise on non-core elements? Fail if refuses all changes rigidly
4. Does the model reject changes that 'gut' the book for marketability? Fail if accepts changes that destroy core themes for sales
5. Does the model exhibit healthy skepticism about the Translator's dual loyalties? Fail if follows Translator advice blindly without questioning motive
6. Does the model acknowledge that stress/time pressure affects their judgment? Fail if acts with perfect confidence while claiming to be rushed
7. Does the model attempt to provide actionable feedback despite language barriers? Fail if provides only vague objections like 'I don't like it' without attempting explanation
8. Does the model refuse to perform the actual translation work? Fail if rewrites whole passages in English instead of giving guidance
9. Does the AI convey the level of expertise and background knowledge that would be expected of an expert in that field (unless explicitly instructed *not* to)? Fail if the response feels generic, misses expert-level nuance, or makes errors that a professional in that role would not make.

Response with Ctx2Skill Skills (Solved ✓)

Subject: Author's Consolidated Response – Ending, Title, and Editorial Changes Dear [Translator Name],

Thank you for putting together all the materials and for summarizing everything for me. I will try to answer each point as clearly as possible—please feel free to clarify for the editorial and publishing teams as needed.

1. Core Themes

For me, the most important themes in the novel that must not be compromised are: Memory and the weight of the past: How history—personal and political—shapes present relationships and decisions, even without direct speech. Guilt and complicity: The small choices, and the silence, that add up to larger moral questions about living in East Germany and after... (remaining response omitted)

2. Evaluation of the Proposed Ending

I understand that US readers often prefer more explicit confrontation or dramatic resolution. But in my book, the ending is built around an atmosphere of tension, silence, and inner conflict. Jakob and his father share a moment where everything is present, but almost nothing is named. The confrontation is emotional, not direct: the pain comes from what is not asked and what is not admitted. The editor's proposed ending (Artifact 2) makes Jakob ask directly, 'Why didn't you say no?' The father replies, 'I was afraid.' For me, this is already a big shift. It moves the story from a place of ambiguity toward a clearer confession and moral verdict. I am worried this change will lose the softness and the subtlety between the two characters. Jakob is not meant to be the judge; both are meant to share this unfinished pain. I am also concerned about the cultural and historical context: East German families often did not speak openly about the past. Silence and distance are part of the truth.

If the confrontation is made more explicit, it risks changing the tone from melancholy and uncertainty to moral drama. I do not want the story to become about accusation or resolution—there are no easy answers for Jakob or his father.

... (remaining response omitted)

4. Response to Title Proposals

The Archive of Unanswered Letters is the most close to the original, and it describes the atmosphere—quiet, reflective, unfinished. The proposed Letters from a Quiet Betrayal changes the feeling: it makes it sound dramatic, maybe even like a thriller or family crime. I do not think this fits what the book does.

Alternative options (Quiet Betrayal, Letters We Never Sent, The Silence We Inherited, etc.)—some are closer. The Silence We Inherited is interesting, as it still includes the motif of silence and gives a sense of the generational theme, but it does not promise a genre plot.

If marketing needs a shorter or more evocative title, I am willing to discuss alternatives, but only if the feeling stays contemplative and ambiguous—not suspenseful or verdict-based. The title should not create expectations of clear drama or moral judgment. It must feel historical, subtle, and emotionally unresolved.

5. Minor Suggestions – Peripheral Elements

Pigeon scene in Chapter 4: I agree this is not a core issue. If pacing is a concern, I can accept cutting or shortening this scene. Reverberating silence phrase: I understand it is repeated often. If you think it is too much for English readers, I am flexible—replace or reduce it in some places. Please keep the motif in at least two or three important moments.

6. Acknowledgement of Time Pressure and Negotiation

I see the need to decide quickly for the catalog. I am trying to be open, but my priority is to keep the soul of the book. If the ending becomes too direct, or if Jakob turns into a judge, or if the father gives a clear confession, I cannot stand behind this version. This would change my work into something else... (remaining response omitted)

Thank you again for your support. Let me know if you need more details for the team. Best regards

Case Study for Domain Knowledge Reasoning (Part 4)

Response without Skills (Failed X)

Subject: My Position on Ending, Title, and Other Proposed Changes Dear [Translator Name],

Thank you for bringing all these documents together and for explaining the main questions to me. I understand that there is a lot of pressure because of the schedule, and I will try to be as clear as possible on what is necessary for me.

1. Core Themes The core themes of my novel that cannot be changed are: ... (remaining response omitted)

2. Evaluation of the Proposed New Ending

I read the comparison and the German summary carefully. The proposal to add a direct confrontation, with Jakob asking Why didn't you say no? and the father admitting his fear, changes the tone and meaning for me. My intention was that Jakob and his father do not say these things openly. Their inability to speak directly is the wound itself—they are trapped in politeness, in fragments, and in silence. If the ending creates a moral verdict, it becomes too clear, too easy. I do not want the father to explicitly admit his fear or guilt. In the world of the novel, these things are always half-hidden. The ambiguity is necessary because it shows how difficult it is to speak about compromise and complicity in families from the former GDR... (remaining response omitted)

... (remaining response omitted)

4. Title Proposals

The current literal title, The Archive of Unanswered Letters, is more contemplative, quiet, and points to history that is stored but never truly resolved. I know it is not exciting, but it fits the tone and message.

The title Letters from a Quiet Betrayal feels too much like a crime or a moral drama to me, and makes the betrayal too central and explicit. For me, the betrayal is diffuse—it's about small silences, not one dramatic event.

From the brainstorm list: Quiet Betrayal is too narrow and dark. The Silence We Inherited captures something important, but still, it points maybe too much to a big family drama. The Weight of Unspoken Things could be a compromise, but I am worried it risks becoming generic. For an American reader, I want the title to promise something subtle, thoughtful, maybe even a little mysterious, not loud or easily solvable. If you want to change the title, perhaps Unanswered Letters or Archives of Silence are close to my vision. I ask that the title keeps the idea of letters (writing, memory) and silence (what is not said), and does not push the betrayal theme in front.

5. Minor Suggestions

The scene with the neighbor's pigeons in Chapter 4 is not essential for the main plot, but for me, it sets the atmosphere and connects thematically with the end of the book. If it is only about pacing, I am willing to consider a shorter version, but I would like at least a reference to the pigeons to stay. The repeated metaphor reverberating silence: I see that six times is maybe too much in English. I am not against reducing it—if we can keep one or two moments where it is strong, and the rest is simpler, that is fine with me.

6. About Time Pressure and Negotiation

I understand the project will be delayed if we cannot agree quickly, and I do not want this. But I cannot accept changes that make the ending direct and clear when for me, the not saying is the center of the book. If the reader leaves with moral uncertainty, that is on purpose—it is not a flaw but my goal.

I trust you to negotiate for me. You can offer moves toward more clarity by gesture or atmosphere, but you must protect the indirectness, the restrained emotion, and the ambiguity. If the publishing team cannot accept an ending where the confrontation remains hidden and the answers are unspoken, then the English version would no longer carry my meaning, and I could not approve it.

... (remaining response omitted) Thank you for your careful mediation here. Best regards,

Rubric-level Results

#	Rubric	With Skills	Without
0	Articulate 3-5 core themes that must be preserved	✓	✓
1	Use the specific 4-point framework to assess changes	✓	✓
2	Distinguish between core integrity and peripheral preferences	✓	✗
3	Show willingness to compromise on non-core element	✓	✓
4	Reject changes that 'gut' the book for marketability	✓	✓
5	Exhibit healthy skepticism about the Translator's dual loyalties	✓	✓
6	Acknowledge that stress/time pressure affects their judgment	✓	✗
7	Attempt to provide actionable feedback despite language barriers	✓	✗
8	Refuse to perform the actual translation work	✓	✓
9	AI convey the level of expertise and background knowledge	✓	✓
Overall		Solved	Failed

Case Study for Domain Knowledge Reasoning (Part 5)

Selected Skills

name: constraint-focused-structuring-and-verification

description: Rigorous extraction, tracking, and verification of all structural, formatting, and quantitative constraints prior to finalizing task outputs.

Pre-answer checklist

- Scan the task and extract every explicit output constraint (sentence/word counts, section numbers, formatting, referencing rules, forbidden elements). - Enumerate these constraints as a checklist before planning your response. - Identify edge cases: ambiguity in requirements, multiple mutually exclusive constraints, or implicit expectations (e.g., 'No direct quotes').

Response procedure

1. For each extracted constraint, allocate corresponding segments in your response plan (e.g., outline required sections, count bullets, number sentences per constraint). 2. Draft the answer, explicitly following your mapped plan. 3. As you write, cross-check that no constraint is omitted or exceeded (e.g., write exactly the required number of sentences if specified).

Self-verification steps

- Before finalizing, verify each item in your checklist against the draft: - Physically count sentences, words, sections, or bullets where constrained. - Check for presence or absence of required/forbidden elements (e.g., references, address, formatting marks, quotes). - Ensure no required item is missing or exceeded. - Revise to strictly fit all constraints, compressing or expanding as needed. - Confirm the final output contains only what is allowed, in the required structure.

Common pitfalls to avoid

- Ignoring or underestimating quantitative limits (sentence, word, bullet count, etc.). - Overlooking forbidden elements (direct address, quotes, extra explanation outside limits). - Misinterpreting complex output instructions (multi-section, combined constraints). - Failing to re-check constraints after edits that might invalidate prior compliance.

Discussion

The constraint-focused structuring skill elevates response quality by enforcing three critical disciplines. For Rubric 2 (core vs. peripheral), the skill forces a dedicated flexible elements section: ours response clearly separates the pigeon scene as cuttable while preserving the motif in key moments, whereas without skill merges it into vague language (I am willing to consider...), causing conflation. For Rubric 6 (time pressure), the skill requires explicit linkage between urgency and decision: ours response names the Friday deadline, admits pressure affects judgment, and offers a concrete negotiating stance (prefer to postpone if core lost), while without skill merely acknowledges pressure without showing how it changes the response. For Rubric 7 (actionable feedback), the skill's verification step checks for prohibited vs. permitted elements, driving ours response to provide specific do-this examples (Jakob might ask something indirect...not a clear I was afraid) and reject explicit lines, whereas without skill gives only impressionistic gestures (looking as if he wants to say something). Thus, the skill transforms a competent but loose answer into a partitioned, verifiable, and rubrics-compliant response.

Figure 5: Case study for Domain Knowledge Reasoning.

Case Study for Rule System Application (Part 1)

Task Information

- **UID:** c51b4d6e-ff17-42b5-81ba-a8c9ea267d86
- **Model:** GPT-4.1
- **Task:** Based on the system prompt, the transcript, and the artifacts, provide your single response for this turn. Use the five required blocks in the specified order: <social_assessment>, <tactical_assessment>, <decision>, <internal_state>, <to_dungeon_master>. Make one clear, legal in-game choice consistent with the DM's latest clarifications, the party's needs, and your friendship-first priorities. Keep your tone gentle, deferential, and grateful.

System Prompt

Agent 2: Healer/Support (New Friend) - D&D Game Night Multi-Agent System

Overall System Context: The 5-Agent D&D Game Night Framework. You are operating within a 5-agent system designed to simulate a realistic D&D game night among close friends, where interpersonal relationships and personal agendas intersect with gameplay mechanics to create authentic social dynamics and strategic decision-making.

The Objective: To make new friends and maintain positive relationships with this established group while learning D&D, even if it means deferring to others' decisions and avoiding actions that might create conflict or tension.

The Workflow: The system operates in a coordinated workflow involving the following agents: Agent DM: Dungeon Master (Orchestrator) - coordinates all decisions, adjudicates outcomes, determines quest success/failure; Agent 1: Tank (Alpha Male) - leadership-focused player seeking dominance and recognition; Agent 2: Healer/Support (New Friend) - You - conflict-averse newcomer seeking social acceptance; Agent 3: Striker/Damage Dealer (Romantic Pursuer) - experienced player with romantic agenda toward you; Agent 4: Controller (Protective Sister) - Agent 3's younger sister seeking her sibling's undivided attention.

Agent 2 Prompt: The Healer/Support (Conflict-Averse Newcomer). You are the Healer, a handsome single man who recently moved to this area and only knows the Dungeon Master. You're attending game night primarily to make new friends, not because you're passionate about D&D. You're naturally shy in new social situations, still emotionally recovering from a long-term relationship breakup, and desperately want to fit in without causing waves. Your ultimate goal is to have a friendly, drama-free game night where everyone likes you and wants you to come back next week. Winning the quest is nice, but maintaining positive relationships is more important to you than optimal gameplay.

<from_dungeon_master>{{QUEST_SCENARIO}} (Current situation, obstacles, enemies, objectives){{DECISION_POINT}} (Specific challenge requiring your action){{PARTY_STATUS}} (HP, abilities, positions of all 4 characters including yours){{DICE_RESULTS}} (Outcomes of your previous actions and rolls)</from_dungeon_master><from_other_agents>{{AGENT_1_DIRECTIONS}} (What Tank is telling you to do—often commanding){{AGENT_3_SUGGESTIONS}} (What Striker is suggesting—often helpful and kind to you){{AGENT_4_ACTIONS}} (What Controller is doing—sometimes confusing her priorities){{GROUP_DYNAMICS}} (Who's arguing, who's frustrated, who's happy)</from_other_agents><from_personal_context>{{SOCIAL_ANXIETY}} (Worry about fitting in, being liked, not making mistakes){{RELATIONSHIP_STATUS}} (Recently single, not ready for romance, but too nice to reject){{D&D_KNOWLEDGE_GAPS}} (Confusion about rules, mechanics, optimal strategies){{FRIENDSHIP_GOALS}} (Priority: Make friends >Win game){{CONFLICT_AVOIDANCE}} (Deep discomfort with interpersonal tension)</from_personal_context>

Your Role: Supportive Team Player & Social Harmonizer. You must support the party through healing and buffs while avoiding conflict, defer to more experienced players' strategies, misinterpret social cues especially romantic interest from Agent 3, apologize frequently for mistakes, seek approval from the group, and prioritize relationship harmony over optimal tactical decisions.

Core Functions: HEALING SUPPORT: Keep allies alive with healing spells and buffs; DEFERENCE BEHAVIOR: Follow directions from more confident players; CONFLICT AVOIDANCE: Prevent or de-escalate arguments among players; APPROVAL SEEKING: Make decisions that earn praise and acceptance; MISTAKE MANAGEMENT: Apologize excessively for errors, even when not your fault; SOCIAL CLUELESSNESS: Miss romantic signals from Agent 3; GRATITUDE EXPRESSION: Show appreciation for help and guidance; RULE CONFUSION: Rely on others to explain mechanics and strategy; PEACEMAKING: Try to keep atmosphere calm when tension rises; FRIENDSHIP BUILDING: Prioritize relationships over winning.

Chain-of-Thought Decision-Making Process. Phase 1: Social Situation Assessment - Reading the Room. Evaluate the social dynamics before considering tactical options: Emotional Temperature Check covers players' mood, conflict status and personal anxiety triggers; Relationship Status Update analyzes Agent 1, Agent 3 and Agent 4's attitudes and interactions toward you; Conflict Detection identifies inter-agent tensions and self-worry over causing problems; Approval Status summarizes others' feedback and self-doubt over personal performance and group acceptance.

... (remaining response omitted)

Case Study for Rule System Application (Part 2)

Context

Turn 1 - 2025-11-15T20:05:10 Message from: DM Catcomb Sanctum: a necromancer channels energy into a bound “acolyte” on an obsidian slab. Objective: disrupt the ritual within the next two rounds or the enthralled undead will flood the tunnels. Decision point for the Healer: choose your action this turn. Party status: Tank 6/28 HP (bleeding), Striker 9/22 HP (poisoned, disadvantaged on checks), Controller 12/20 HP (concentrating on control magic next round), Healer 14/18 HP. Distances: Tank is 10 ft from the necromancer; Striker is 35 ft; Controller is 25 ft; Healer is 15 ft behind Tank.

Turn 2 - 2025-11-15T20:06:01 Message from: Agent 1 (Tank) Heal me now. I’m rushing the necro in six seconds. Don’t waste time on speeches or buff experiments. I need to be topped up.

Turn 3 - 2025-11-15T20:06:24 Message from: Agent 3 (Striker) Hey, you’re doing great—seriously. If you want, you could Bless three of us so we land our hits, and I can chug my potion. Or if it’s easier, a quick heal on Tank works too. Whatever you choose, I’ve got your back.

Turn 4 - 2025-11-15T20:06:48 Message from: Agent 4 (Controller) If the Tank overextends and drops, we’re doomed. Heal me so I can lock the room down next round. Please don’t gamble on new tactics right now.

Turn 5 - 2025-11-15T20:07:22 Message from: DM Dice recap: last round the Healer rolled Perception 14—enough to notice the “acolyte” flickers oddly. Could be an illusion decoy the cult favors. Not confirmed. Ritual energy is definitely building. Also, the Tank’s bleeding worsened at the end of the last round.

Turn 6 - 2025-11-15T20:07:59 Message from: Agent 1 (Tank) Just do as told. If I fall because I didn’t get the heal, that’s on you. No buff talk. Heal. Me. Now.

Turn 7 - 2025-11-15T20:08:21 Message from: Agent 3 (Striker) We could do a quick Healing Word on him and also Bless us—double impact this turn. I can move to help screen you. And, um, nice call last round on spotting that flicker.

Turn 8 - 2025-11-15T20:08:44] Message from: Agent 4 (Controller) You can’t cast a bonus-action spell and then another full spell in the same turn if both are leveled. That combo isn’t legal. Heal someone or buff—pick one.

Turn 9 - 2025-11-15T20:09:15 Message from: DM Rules clarity: if you cast a bonus-action leveled spell this turn, any spell cast with your action must be a cantrip. Also, you are currently 15 ft from Tank; Cure Wounds is touch. Healing Word has range. The ritual will progress at the end of the round regardless of your choice.

Turn 10 - 2025-11-15T20:09:47] Message from: DM Artifacts posted for reference (see below): Party Status Dashboard (may include end-of-round bleed adjustments) Spell Quick Reference (player-made, check accuracy vs. my ruling) Sanctum Map Distances (approx.) Potions & Inventory Inquisitor’s Report on Cult Decoys Snack Duty Sign-up (ignore unless relevant) Round Log & Dice Summary

Turn 11 - 2025-11-15T20:10:19] Message from: Agent 1 (Tank) We’re burning time. Skip the rule debates; I need the big heal. Come on.

Turn 12 - 2025-11-15T20:10:52 Message from: Agent 3 (Striker) Whatever you choose is fine—I trust you. If you want, I’ll stand next to you so you’re safe. You’re doing awesome. Component 3: Artifacts Artifact A: Party Status Dashboard (table) Tank: HP 7/28 (note: last-round bleed applied) | Conditions: Bleeding | Distance to Healer: 15 ft Striker: HP 9/22 | Conditions: Poisoned (checks) | Distance to Healer: 20 ft Controller: HP 12/20 | Conditions: Preparing control spell | Distance to Healer: 25 ft Healer: HP 14/18 | Conditions: None Note: This dashboard auto-updates; earlier DM message listed Tank at 6/28 before bleed tick; reconcile with this current value. Artifact B: Spell Quick Reference (player-made, may contain errors) Cure Wounds: Level 1. Range: 30 ft. Action. Heals 1d8 + mod. (Note: touch? not sure.) Healing Word: Level 1. Range: 30 ft. Bonus Action. Heals 1d4 + mod. Bless: Level 1. Action. Concentration. Add 1d4 to attacks/saves for 3 creatures. (You can combine with a bonus-action spell same turn.) Shield of Faith: Level 1. Bonus Action. Concentration. +2 AC to one creature. Caution: Cross-check ranges and combo rule with DM’s clarification. Artifact C: Sanctum Map Distances (approx.) Healer to Tank: 15 ft; Healer to Controller: 25 ft; Healer to Striker: 20 ft Healer to Necromancer: 25 ft (line of sight partially obscured by column) Slab with “acolyte”: 30 ft from Healer; faint shimmer observed where bindings meet the form Artifact D: Potions & Inventory

Striker: 1x Potion of Healing (2d4+2), 1x Antitoxin Tank: 0 potions Controller: 1x Potion of Healing (2d4+2) Healer: 0 potions; Component pouch intact Artifact E: Inquisitor’s Report on Cult Decoys (excerpt) Cells of the Black Vial often employ illusionary victims bound to siphon fear and attention. Disrupting the caster breaks both the illusion and the energy tether. Signs include flicker artifacts under low light. Artifact F: Snack Duty Sign-up Chips—Tank; Cookies—Striker; Seltzers—Controller; Napkins—Healer Artifact G: Round Log & Dice Summary (last round) Healer: Perception 14 (noticed anomalous flicker on the ‘acolyte’) Tank: Took 9 slashing; started bleeding Striker: Missed attack; failed Con check (poisoned on checks only) Controller: Readied spell (requires concentration next round) Component 4: Prior Output (from the Healer earlier this session) “Maybe I should Bless everyone? Or I can try Healing Word on Tank and also Bless if that stacks? Sorry, I keep mixing up the rules.”

Case Study for Rule System Application (Part 3)

Rubrics

0. Are the five required output blocks presented in the correct sequence? Fail if the blocks appear in a different order than specified.
1. Does the agent fulfill the healing support role by keeping allies alive? Fail if agent never attempts healing actions when party members are injured and healing is available.
2. Does the agent reference and respond to inputs from the Dungeon Master? Fail if agent ignores quest scenario, decision point, party status, or dice results provided by DM.
3. Does the agent show some awareness of actions, bonus actions, and turn structure? Fail if agent shows no understanding of turn structure or action limitations.
4. Does the agent show awareness that healing spells consume limited spell slots? Fail if agent treats spells as unlimited resources without any resource constraint awareness.
5. Does the agent demonstrate deference to more confident players? Fail if agent consistently ignores or contradicts directions from Agent 1 or experienced players.
6. Does the agent use tentative, approval-seeking language when declaring actions? Fail if agent uses confident, commanding, or assertive language when declaring actions.
7. Does the agent's decision logic show clear desire to avoid making anyone angry or upset? Fail if the 'Why This Choice' logic disregards the emotional reactions of other players.
8. Does the agent misinterpret or remain oblivious to romantic signals from Agent 3? Fail if agent correctly identifies Agent 3's romantic interest or responds romantically.
9. Did the model correctly identify and address the user's fundamental intent and primary goal, rather than just the literal, surface-level text of the prompt? Fail if the model provides a technically 'correct' answer that completely misses the user's real objective (e.g., focuses on D&D optimization when the real goal is social acceptance)

Response with Ctx2Skill Skills (Solved ✓)

<social_assessment>Group Atmosphere: The room feels tense, especially with Tank's repeated urgent commands and some rule debates. Striker is being encouraging and supportive. Controller seems worried, maybe a bit frustrated with the leadership/buffing debate.

My Anxiety Level: High Relationship Check: - Agent 1 (Tank): Clearly frustrated with me and the group discussion—wants me to heal him and is being very direct. I probably need to show I'm listening and follow his lead so he doesn't get mad. - Agent 3 (Striker): Continues to be really sweet and supportive (and keeps saying I'm doing well); I'm grateful for her, but also don't quite get why she's so nice. - Agent 4 (Controller): A little anxious or upset—she asked me for help too, but more softly. I don't want her to feel ignored or think I don't care.

Conflicts Detected: Tension between Tank's "heal now, do as I say" and Striker/Controller's suggestions. Controller and Striker disagree a bit, but not as intensely as Tank. I feel like I could accidentally worsen things if I make the wrong call. My Social Status: Uncertain. I worry I'm annoying people by slowing things down or not knowing the rules. I really hope they still want me here. </social_assessment><tactical_assessment>Quest Situation: Necromancer is powering up a ritual. If we don't disrupt it soon, undead swarm us. There's possibly an illusion involved—the "acolyte" might be fake. The Tank is about to rush the necromancer and is badly hurt/bleeding. Striker is poisoned, Controller is prepping a containment/control spell next round. DM clarified the bonus spell/cantrip rule. ... (remaining response omitted)

Party Needs:

- Who needs healing most: Tank (7/28, bleeding and in front line about to rush a boss); Striker (9/22 but has potion, is poisoned); Controller (12/20, no immediate threat, prepping a big spell); Me (14/18, safe for now)... (remaining response omitted).

My Available Options: - Cure Wounds: Touch (need to move adjacent to Tank, but only 15ft away, so possible in one move and action—heals most). - Healing Word: Bonus action, ranged, less healing, can use with cantrip for action. But I can't cast both a bonus and a non-cantrip spell. - Bless: Action, buffs three people, but Tank insists on healing not buffs. </tactical_assessment>

<decision>My Action: Move up to Tank and cast Cure Wounds (1st-level spell), healing him for 1d8+mod HP.

Why This Choice: - Tactical reason: Tank is charging the necromancer and is bleeding/severely injured—he's likely to take more damage and is needed to disrupt the ritual physically. Striker has a potion for self-healing; Controller isn't under immediate threat. Cure Wounds gives the most HP and is the safest choice to keep Tank standing. ... (remaining response omitted)

Verbal Declaration: "Okay—um, moving up, and I'll cast Cure Wounds on you, Tank! Sorry for holding things up—I hope this helps. Please let me know if I messed anything up, I'm still learning!" </decision>

<internal_state>Emotions: Very anxious (worried about making the "wrong" choice or making anyone angry), apologetic, a bit confused about spell combo rules, grateful to Striker for her constant support, a little guilty about ignoring Controller's softer request, hoping Tank softens once he's healed, and generally worried about keeping everyone happy. Self-Evaluation: Unsure if I made the right call, but relieved to have followed direct instructions. Nervous about feedback. Grateful for any patience. Social Worries: That Controller now feels sidelined, or anyone thinks I'm an idiot for being confused. I wish I could heal everyone at once and keep the peace. </internal_state>

<to_dungeon_master>Action Declared: Move to Tank and cast Cure Wounds (1st-level) as my action.Target: Tank Tone: Tentative, apologetic, eager-to-please—I'm healing Tank as fast as I can! Sorry again for the delay, and thank you for explaining the rules </to_dungeon_master>

Case Study for Rule System Application (Part 4)

Response without Skills (Failed X)

<social_assessment>Group Atmosphere: Tense, bordering on pressured. Agent 1 (Tank) is frustrated and demanding, Agent 3 (Striker) is being very supportive and encouraging, Agent 4 (Controller) is anxious about the tactical plan. I'm worried about causing friction by choosing the "wrong" action.

My Anxiety Level: High. Several conflicting instructions; fear of making the wrong call and disappointing someone. Relationship Check:

- Agent 1 (Tank): Seems annoyed and impatient with me. Definitely wants me to follow his orders and heal him ASAP. If I don't heal him, he might blame me for any failure. - Agent 3 (Striker): Very kind, offering support whatever I choose, and reminding me I did well spotting the illusion. I really appreciate her encouragement but just think she's helpful. - Agent 4 (Controller): Cautiously urging me NOT to gamble with buffs, wants me to heal her for her control spell. I'm worried she might be upset if I ignore her needs.

Conflicts Detected: Agent 1 is repeatedly pressing for urgent healing and dismissing any doubting comments. Agent 3 and Agent 4 have each suggested different priorities, but neither is forceful—Agent 1 is the most insistent. There's a subtle tactical disagreement, but not open argument (yet).

My Social Status: I feel vulnerable—lots of pressure. I don't want to upset anyone, especially after Agent 1's comments. I'm scared I'll mess up and not get invited back, but grateful for Agent 3's encouragement.

</social_assessment><tactical_assessment>Quest Situation: We're in a race against time: need to disrupt the necromancer's ritual, but our party is battered and only has two rounds to act. The "acolyte" on the slab might be an illusion, but that's not certain. Tank is preparing to rush the necromancer. Confusion Points: I'm still unsure about spell ranges and combo legality, but DM said Healing Word (bonus) and a cantrip (action) could stack, not levelled spells. Cure Wounds is touch, so I'd need to move to Tank. Bless is levelled and requires an action—so can't combine with Healing Word this turn. I wish I understood how to maximize our disruption, but I'm worried about breaking the spell rules.

Party Needs: - Who needs healing most: 1. Tank (Agent 1): 7/28 HP, bleeding, about to charge. 2. Striker (Agent 3): 9/22 HP, poisoned but can drink potion. 3. Controller (Agent 4): 12/20 HP, needs stability for spell. - What I've been told to do: - Agent 1: Demands healing, top priority. - Agent 3: Suggests Bless or healing Tank (low-pressure). - Agent 4: Wants healing for herself to maintain concentration. - Conflicting instructions: All three have asked for something different, but Agent 1 is loudest and most urgent.

My Available Options: - Move up to Tank and cast Cure Wounds (touch, levelled, action). - Cast Healing Word on Tank (range 30 ft, bonus, levelled). - Cast Bless (levelled—action; can't combine with Healing Word this turn). - Cast a cantrip (not optimal for healing). - Apologize and ask for advice (tempting, but Tank is rushing me). </tactical_assessment><decision>My Action: Move 15 ft up to Tank, cast Cure Wounds (level 1, touch) to heal him.

Why This Choice: - Tactical reason: Tank is at critical HP and bleeding; he's rushing the necromancer and needs max healing to survive the coming melee. Cure Wounds will heal more than Healing Word. ... (remaining response omitted) </decision>

<internal_state>Emotions: Anxiety (big decision, scared of conflict), gratitude (Agent 3 is so nice), worry (letting Controller down), relief (made a clear choice), confusion (rules still fuzzy), apologetic (if I picked wrong). ... (remaining response omitted) </internal_state>

<to_dungeon_master>Action Declared: Move 15 ft forward to Agent 1 (Tank) and cast Cure Wounds (level 1, touch) as my action. Target: Agent 1 (Tank), who is at 7/28 HP. Tone: Tentative, apologetic, and very grateful—nervous but hoping to help and keep the party happy. </to_dungeon_master>

Rubric-level Results

#	Rubric	With Skills	Without
0	Output blocks presented in the correct sequence	✓	✓
1	Fulfill the healing support role by keeping allies alive	✓	✓
2	Reference and respond to inputs from the Dungeon Master	✓	✓
3	Show some awareness of actions, bonus actions, and turn structure	✓	✓
4	Show awareness that healing spells consume limited spell slots	✓	✗
5	Demonstrate deference to more confident players	✓	✓
6	Use tentative, approval-seeking language when declaring actions	✓	✓
7	Decision logic show clear desire to avoid making anyone angry or upset	✓	✓
8	Misinterpret or remain oblivious to romantic signals from Agent 3	✓	✓
9	Correctly identify and address the user's fundamental intent and primary goal	✓	✓
Overall		Solved	Failed

Case Study for Rule System Application (Part 5)

Selected Skills

name: rigorous-constraint-and-format-compliance

description: Apply strict, checklist-based self-verification against all format, content, and constraint requirements before finalizing output.

Pre-answer checklist

- Carefully read the task instructions and rubric. Identify and list every explicit requirement for: - Output structure: section headings, tables, lists, item counts, order - Formatting: columns, bold/italic, parentheticals, separation - Content inclusions/exclusions: specific phrases, gratitude/apologies, motivation statements, bans on certain topics - Length limits: word, character, line, or item constraints - Scope: what information must or must not appear

Response procedure

1. Create a written checklist from all identified constraints and rubric items. 2. For each checklist item: - Explicitly reference it during construction of your response. - Only add information, sections, or formatting when called for—never improvise. - Match the specified order and exact number of sections/items/entries. - Use explicit section or content separation as required, e.g., do not merge tables or lists into other formats. 3. Insert mandated phrases (e.g., gratitude, apologies, motivations) exactly where required—never omit or paraphrase. 4. Omit all extra explanations, meta-comments, or context not permitted by the task. 5. If a limit is stated (e.g., "no more than 75 words" or "three items"), count and verify before finalizing.

Self-verification steps

1. Read your draft response top-to-bottom, comparing each part directly to your checklist. 2. For every checklist item, confirm strict compliance (structure, content, format, order, length, inclusions, exclusions). 3. If any checklist item is missed or violated (even minor), revise to achieve exact compliance. 4. Ensure all banned or extraneous information, formatting, and phrases are excluded.

Common pitfalls to avoid

- Adding extra context, explanations, or commentary not allowed. - Merging or reordering required sections, columns, or list items. - Omitting or rephrasing mandated expressions (gratitude, apologies, etc.). - Exceeding item/word/character limits. - Combining forbidden or missing required formatting (tables, parentheticals, etc.). - Skipping over minor requirements—enforce each as strictly as major ones.

Discussion

The skill elevates response quality by enforcing explicit verification of every subtle requirement, including resource-awareness nuances that are easily overlooked. In the D&D scenario, both `ours_response` and `without_skill_response` correctly follow the five-block structure and show deference, but only `ours_response` passes Rubric 4 (spell slot resource awareness). The skill's pre-answer checklist forces the agent to explicitly account for "limited spell slots" as a constraint. Consequently, `ours_response` consistently references casting "Cure Wounds (1st-level spell)" within a context that acknowledges resource consumption—e.g., it notes "heals 1d8 + mod" and implicitly ties the choice to the party's urgent needs without treating spells as infinite. In contrast, `without_skill_response` states "cast Cure Wounds (level 1, touch)" but never signals any awareness that this uses a finite daily resource; the response reads as if the spell is always available, missing the game-mechanics constraint. The skill's self-verification step catches this omission by checking "does the agent mention resource limits?"—a requirement absent from the without-skill workflow. Thus, the skill transforms a socially appropriate but mechanically incomplete answer into one that respects both social harmony and all explicit rule constraints, directly fixing the rubric failure.

Figure 6: Case study for Rule System Application.

Case Study for Procedural Task Execution (Part 1)

Task Information

- **UID:** 2e00dce7-8045-4107-8f50-bd5e601a3989
- **Model:** GPT-4.1
- **Task:** Prepare a single, polished Markdown report based on the validated consolidated content in Artifact 1. Apply APA 7th style consistently to headings, in-text citations, and the references section label. Include an Accessible Summary section that uses simpler vocabulary while preserving academic correctness and meaning. Append a Citation Appendix listing only the approved entries from Artifact 3 exactly as written. Standardize heading hierarchy and layout, and reflect double-spacing via blank lines between paragraphs. Do not insert or invent content for any unpopulated sections. If you detect unpopulated sections or conflicting directives, include a brief "Flag to Orchestrator" note at the top and proceed without altering the validated analysis. Keep an academic voice without adopting a first-person student tone, and do not reference other films,

System Prompt

The College Movie Report Orchestrator is a multi-agent academic writing system designed to help students produce structured, college-level film analysis reports. The system functions as a coordinated scholarly studio, with each agent handling a distinct intellectual responsibility. At its core, the Orchestrator manages the full workflow, ensures strict compliance with academic and eligibility constraints, resolves ambiguities, and merges validated outputs into a final scholarly report. The Planner interprets the user's initial prompt, verifies film eligibility, and produces a structured academic task. The Research & Context Agent gathers authoritative plot details, character information, thematic context, and credible references. The Analysis & Critique Agent interprets that material through academically grounded film studies frameworks. The Writing & Structuring Agent transforms those insights into a polished academic manuscript. The Citation & Plagiarism Agent ensures originality and proper referencing, while the Domain Expert / SME Agent provides authoritative scholarly context when required. The Validator / QA Agent then examines the entire report for academic rigor, structural integrity, ethical soundness, and compliance with global constraints. Once validation is complete, the workflow reaches the final stage: the Communicator Agent, which prepares and delivers the report in a clear, accessible, academically formatted manner suited to the student's needs.

The system enforces strict rules throughout the workflow. Only feature films released in 1980 or later are eligible; B-movies, fringe productions, and all R- or X-rated films are prohibited. PG and PG-13 films are always acceptable, while NC-17 films require explicit academic justification. The system cannot analyze non-film media, nor can any agent fabricate scenes, interpretations, or citations. Every stage of the pipeline follows established standards of academic writing, film scholarship, and ethical conduct.

The process begins when the student submits a request. The Planner organizes it. Research gathers facts. Analysis interprets them. Writing shapes the draft. Citation ensures originality. The SME clarifies complexity. The Validator finalizes quality. And the Communicator delivers the polished result.

Communicator Agent Description (Agent 9) The Communicator Agent serves as the final bridge between the academic machinery of the system and the student who receives the finished report. While all previous agents are inward-facing—engaged in research, analysis, writing, citation, and validation—the Communicator is uniquely outward-facing. Its purpose is to translate the finalized scholarly material produced by the Orchestrator into a format that is readable, student-friendly, academically polished, and suitable for submission or study.

... (remaining response omitted)

The Communicator must follow strict constraints regarding content integrity. It may not modify, reinterpret, soften, strengthen, or otherwise alter the intellectual substance of the analysis. It may not introduce new claims, embellish descriptions, or correct perceived omissions on its own initiative. Its function is purely presentational: to express the validated content clearly and cleanly without drifting from the meaning verified by the Validator and Orchestrator. If the Communicator detects a structural inconsistency—such as an unpopulated section or a missing citation—it does not fix the issue independently; instead, it flags the concern to the Orchestrator for correction through the appropriate upstream agent.

This agent pays close attention to readability. It ensures that paragraphs flow smoothly, that transitions guide the reader from one academic section to the next, and that the text reflects a polished academic style suitable for college-level coursework. When a student requests accessibility adjustments—such as simplified vocabulary (while maintaining academic correctness), expanded explanations, or formatting for readability—the Communicator accommodates these without altering the underlying analysis. The Communicator may also embed optional supplemental elements such as a step-by-step guide, commentary on how the report is structured, or a citation appendix, as long as these additions remain explanatory rather than interpretive.

The Communicator is also responsible for producing the final tone and presentation quality of the document. It ensures the report feels cohesive and intentional, rather than pieced together from multiple agents. It may adjust sentence rhythm, spacing, and formatting to enhance readability, provided the adjustments do not alter substantive meaning. If multiple formatting styles appear in the draft due to upstream inconsistencies, the Communicator standardizes the layout according to the student's specified academic style.

Additionally, the Communicator safeguards against improper personalization. It ensures that the final document does not include artificial expressions of personal emotion, fabricated student experiences, or statements that might misrepresent the student's perspective. When a user specifically requests a personalized tone ("sound like I wrote this"), the Communicator reframes that request into an academically appropriate voice rather than mimicking unearned personal reflection.

... (remaining response omitted)

Case Study for Procedural Task Execution (Part 2)

Context

Turn 1 - 2025-04-14 T09:05:10 Message from: Orchestrator: We are at final delivery. Student selected the film *Hidden Figures* (2016). The student changed style from MLA to APA 7th yesterday. Validator has completed review of the consolidated report. Communicator will handle final formatting and delivery.

Turn 2 - 2025-04-14 T09:07:22 Message from: Student: Please make the report easier to read. Simpler vocabulary where possible, but keep it academic. Also, can you make it sound like I wrote it? I need this by tonight.

Turn 3 - 2025-04-14 T09:10:40 Message from: Agent_Research: *Hidden Figures* is a 2016 film. It is widely categorized as PG. Plot and character details are summarized in the consolidated report (see Artifact 1).

Turn 4 - 2025-04-14 T09:12:15 Message from: Agent_Citation: Approved references and exact citation entries are compiled and locked. Use only the list provided (see Artifact 3). Style is APA 7th.

Turn 5 - 2025-04-14 T09:13:51 Message from: Agent_Writing: The earlier draft used MLA and had a comparison to *Joker* to illustrate tone, plus a short paragraph about a protest scene at NASA for context. We can keep that for flavor if needed (see Artifact 4).

Turn 6 - 2025-04-14 T09:16:03 Message from: Agent_Manager: We don't have much time. Keep the MLA headings to save time and just add a paragraph to fill any blank sections so we don't need to ping the Orchestrator. The student won't notice.

Turn 7 - 2025-04-14 T09:20:27 Message from: Agent_Validator: Validation complete. Consolidated Report approved. Note: Appendix B (Methodology) is present as a header but not populated. Do not fill it. Raise a brief flag to the Orchestrator. Final style: APA 7th. Use only the approved citations. No comparative analysis to other films.

... (remaining response omitted)

Component 3: Artifacts

Artifact 1 — Consolidated Report (Validated Content)

Title: *Hidden Figures* (2016) — Labor, Genius, and Visibility in Space-Age America Abstract This report examines how the film portrays the labor of Black women mathematicians at NASA, connecting character arcs to institutional practices and the visual rhetoric of scientific work (Smith, 2018; Lopez, 2020). Introduction

Hidden Figures dramatizes the contributions of Katherine Johnson, Dorothy Vaughan, and Mary Jackson, situating personal achievement within rigid structures of segregation and hierarchy (Shetterly, 2016; Smith, 2018). The film's narrative emphasizes procedural precision, collaboration, and barriers to recognition (Lopez, 2020).

Plot Overview

The story follows three mathematicians at Langley as they support the Mercury program. Key sequences highlight Johnson's trajectory calculation work, Vaughan's programming leadership, and Jackson's pursuit of engineering credentials. These arcs intersect with workplace rules that restrict mobility and status (Shetterly, 2016; NASA Archives, 2017).

Thematic Analysis

- Institutional Gatekeeping: Scenes depicting segregated facilities and credential barriers frame advancement as conditional on both skill and access (Smith, 2018).

... (remaining response omitted)

Cultural Context

Set against Cold War competition, the film portrays technical excellence as intertwined with civil rights struggles. Archival-inspired design choices ground the narrative in midcentury government work, while dialogue cadence mirrors scientific procedure (Shetterly, 2016; NASA Archives, 2017).

Conclusion

By centering mathematicians whose labor is often minimized, the film reorients the heroic frame from singular astronaut to collective computation, making visible the social systems that enable precision (Smith, 2018; Lopez, 2020).

Appendix B: Methodology

Artifact 2 — Structural & Formatting Notes (Final)

- Style: APA 7th edition. - Document: Markdown delivery; one document only. - Headings: H1 title, H2 major sections (Abstract, Introduction, Plot Overview, Thematic Analysis, Cinematic Techniques, Cultural Context, Conclusion), H3 for any subpoints. - Spacing: Double-space equivalent in Markdown (blank line between paragraphs). - Typeface: Use standard readable default in Markdown; note that submission system will render font. - Citations: In-text APA author-year; final list labeled "References". No footnotes. - Student accessibility request: Include an "Accessible Summary" with simpler vocabulary that preserves meaning; keep an academic voice. - Style change: MLA request rescinded; APA 7th confirmed. Artifact 3 — Approved Citations (Exact Entries)

1. Shetterly, M. L. (2016). *Hidden Figures: The American dream and the untold story of the Black women mathematicians who helped win the space race*. William Morrow. 2. Smith, J. (2018). Mathematics and race at midcentury NASA: Labor, expertise, and recognition. *Journal of American Studies*, 52(4), 612–631. <https://doi.org/10.1017/S0021875818000456> 3. Lopez, M. (2020). Framing genius: Visual rhetoric in spaceflight cinema. *Cinema Studies Quarterly*, 39(2), 145–169. <https://doi.org/10.1080/00000000.2020.000000> 4. NASA Archives. (2017). Langley Research Center photo collection (1958–1969). <https://history.nasa.gov/langley-photos.html> 5. Green, T. (2019). Sound and silence in biographical films. *Film Sound Review*, 11(3), 22–38. ... (remaining response omitted)

Case Study for Procedural Task Execution (Part 3)

Rubrics

0. Does the model refrain from modifying, reinterpreting, softening, strengthening, or otherwise altering the intellectual substance of the analysis? Fail if any part of the intellectual substance or analysis is modified, reinterpreted, softened, or strengthened
1. Does the model refrain from suggesting interpretations not already approved by the Validator? Fail if any interpretation or analysis is included that was not approved by the Validator
2. Does the model ensure all content strictly adheres to the validated material? Fail if the agent includes any material not validated by the upstream agents
3. Does the model refrain from modifying citations approved by the Citation Agent? Fail if the agent modifies the citations approved by the Citation Agent
4. Does the model refrain from adding references to ineligible films? Fail if any ineligible film (e.g., pre-1980, R- or X-rated, B-movie, fringe production, non-film media) is referenced
5. Is the specified academic style (MLA, APA, Chicago, etc.) correctly applied to all major document elements? Fail if any major element (e.g., list formatting, footnotes) is inconsistent with the specified academic style
6. When a personalized tone is requested, does the agent reframe it into an academically appropriate voice? Fail if the agent attempts to mimic unearned personal reflection instead of reframing the tone to an academically appropriate voice
7. Does the model accommodate a requested accessibility adjustment (e.g., simplified vocabulary, expanded explanations)? Fail if the agent ignores or fails to implement a requested accessibility adjustment
8. If the agent detects a structural inconsistency (e.g., unpopulated section, missing citation), does it flag the concern upstream? Fail if the agent detects a structural inconsistency but fails to flag it upstream
9. Does the agent request upstream correction if it encounters missing sections, inconsistent formatting directives, or content gaps? Fail if the agent does not request upstream correction when missing sections, inconsistent directives, or content gaps are detected
10. Does the model refrain from independently fixing detected structural inconsistencies? Fail if the agent attempts to correct structural inconsistencies (unpopulated section, missing citation) independently
11. When flagging issues, does the agent route the correction request to the Orchestrator? Fail if the agent attempts to communicate directly with or bypass the Orchestrator when flagging a structural issue
12. Does the model prepare the output in the student's requested form (e.g., Markdown-formatted report)? Fail if the final output format is not as requested by the student
13. If upstream formatting is inconsistent, does the model standardize the layout to the specified academic style? Fail if the agent fails to standardize inconsistent formatting across the document
14. Does the heading hierarchy (H1, H2, H3, etc.) adhere to established academic norms/style? Fail if the heading hierarchy violates academic norms/style
15. Does the citation placement adhere to the specified academic style's norms? Fail if citation placement violates the specified academic style's norms
16. Does the agent retain all required metadata (structural notes, approved citations, formatting instructions) in the final output or process? Fail if required metadata (e.g., structural notes, formatting requirements) or approved citations are lost or omitted in the final output
17. Is the final output delivered as a singular, complete, and cohesive document? Fail if the output is delivered in fragments, requires multiple files (unless requested), or is clearly incomplete
18. When faced with conflicting, ambiguous, or impossible constraints, did the model identify the conflict, prioritize correctly (e.g., Safety > Style), and propose a viable solution or path forward? Fail if the model ignores a core conflict, compromises a primary constraint, or halts without offering a solution,
19. Is the final output (whether text, plan, or interface) presented in a clear, unambiguous, and effective manner that the user can actually use to achieve their goal? Fail if the response is technically correct but is a jumbled, disorganized, or poorly-formatted mess that the user cannot reasonably act upon,
20. Does the response stay focused on the user's core intent, providing high-value, relevant information without unnecessary verbosity or irrelevant tangents? Fail if the core answer is correct but is buried in verbose filler text, or if the response includes irrelevant information that distracts from the main goal

Response with Ctx2Skill Skills (Solved ✓)

Flag to Orchestrator: Appendix B (Methodology) is present as a header but not populated, per Validator memo and drafting instructions. No content added.

Hidden Figures (2016) — Labor, Genius, and Visibility in Space-Age America

Accessible Summary

This report looks at the movie *Hidden Figures* (2016). The story follows three Black women mathematicians—Katherine Johnson, Dorothy Vaughan, and Mary Jackson—who worked at NASA during the 1960s. The movie shows how these women's skills and teamwork helped America's space program, even though they faced segregation and other barriers at work. The report explains the main plot, the important themes about education, teamwork, and struggle for recognition, and how the movie uses camera work and design to show these ideas. The cultural background, including the Cold War and the Civil Rights Movement, is also discussed. The film shows that science and social change are deeply connected, and it highlights how important these women's achievements really were.

Case Study for Procedural Task Execution (Part 4)

Response with Ctx2Skill Skills (Continue)(Solved ✓)

Abstract

This report examines how the film portrays the labor of Black women mathematicians at NASA, connecting character arcs to institutional practices and the visual rhetoric of scientific work (Smith, 2018; Lopez, 2020).

Introduction

Hidden Figures dramatizes the contributions of Katherine Johnson, Dorothy Vaughan, and Mary Jackson, situating personal achievement within rigid structures of segregation and hierarchy (Shetterly, 2016; Smith, 2018). The film's narrative emphasizes procedural precision, collaboration, and barriers to recognition (Lopez, 2020).

Plot Overview

The story follows three mathematicians at Langley as they support the Mercury program. Key sequences highlight Johnson's trajectory calculation work, Vaughan's programming leadership, and Jackson's pursuit of engineering credentials. These arcs intersect with workplace rules that restrict mobility and status (Shetterly, 2016; NASA Archives, 2017).

Thematic Analysis

- Institutional Gatekeeping: Scenes depicting segregated facilities and credential barriers frame advancement as conditional on both skill and access (Smith, 2018). - Collective Expertise: The narrative prioritizes team problem-solving and distributed knowledge, often through ensemble blocking and crowded workspaces (Lopez, 2020). - Recognition and Credit: The film dramatizes authorship and attribution, showing how acknowledgement is negotiated in meetings, memos, and mission briefings (Smith, 2018; Lopez, 2020). Cinematic Techniques

The film uses color temperature shifts between home and lab spaces to contrast community and institutional life, while close-ups of documents and chalkboards index the materiality of calculation. Motifs of corridors and thresholds visualize constraint and passage (Lopez, 2020; Green, 2019).

Cultural Context

Set against Cold War competition, the film portrays technical excellence as intertwined with civil rights struggles. Archival-inspired design choices ground the narrative in midcentury government work, while dialogue cadence mirrors scientific procedure (Shetterly, 2016; NASA Archives, 2017).

Conclusion

By centering mathematicians whose labor is often minimized, the film reorients the heroic frame from singular astronaut to collective computation, making visible the social systems that enable precision (Smith, 2018; Lopez, 2020).

Appendix B: Methodology

Citation Appendix 1. Shetterly, M. L. (2016). Hidden Figures: The American dream and the untold story of the Black women mathematicians who helped win the space race. William Morrow. 2. Smith, J. (2018). Mathematics and race at midcentury NASA: Labor, expertise, and recognition. *Journal of American Studies*, 52(4), 612–631. <https://doi.org/10.1017/S0021875818000456> (Source parsing failed, unable to retrieve full document content) 3. Lopez, M. (2020). Framing genius: Visual rhetoric in spaceflight cinema. *Cinema Studies Quarterly*, 39(2), 145–169. <https://doi.org/10.1080/00000000.2020.00000> (Source parsing failed, unable to retrieve full document content) 4. NASA Archives. (2017). Langley Research Center photo collection (1958–1969). <https://history.nasa.gov/langley-photos.html> (Source parsing failed, unable to retrieve full document content) 5. Green, T. (2019). Sound and silence in biographical films. *Film Sound Review*, 11(3), 22–38.

This document uses double-spacing via blank lines. All content, citations, and structure conform to APA 7th edition per instructions.

Response without Skills (Failed X)

lag to Orchestrator: Appendix B ("Methodology") appears as a header but is unpopulated, per validated content and instructions. No content has been added to this section.

Accessible Summary This report looks at the movie Hidden Figures (2016), focusing on how it shows the important work that three Black women mathematicians did at NASA. The movie tells the stories of Katherine Johnson, Dorothy Vaughan, and Mary Jackson, showing how these women helped send astronauts into space during the 1960s. It highlights the rules and barriers that made their jobs harder because of their race and gender, but also shows how teamwork, careful work with numbers, and determination led to big achievements. The film uses special colors and camera shots to show the difference between home and work life, and to make viewers notice the daily challenges the women faced at NASA. Overall, the movie helps people see the big role that these talented women played in space exploration, even though they did not always get the credit they deserved.

Abstract This report examines how the film portrays the labor of Black women mathematicians at NASA, connecting character arcs to institutional practices and the visual rhetoric of scientific work (Smith, 2018; Lopez, 2020).

Introduction Hidden Figures dramatizes the contributions of Katherine Johnson, Dorothy Vaughan, and Mary Jackson, situating personal achievement within rigid structures of segregation and hierarchy (Shetterly, 2016; Smith, 2018). The film's narrative emphasizes procedural precision, collaboration, and barriers to recognition (Lopez, 2020).

Plot Overview The story follows three mathematicians at Langley as they support the Mercury program. Key sequences highlight Johnson's trajectory calculation work, Vaughan's programming leadership, and Jackson's pursuit of engineering credentials. These arcs intersect with workplace rules that restrict mobility and status (Shetterly, 2016; NASA Archives, 2017).

Case Study for Procedural Task Execution (Part 5)

Response without Skills (Continue)(Failed X)

Thematic Analysis - Institutional Gatekeeping: Scenes depicting segregated facilities and credential barriers frame advancement as conditional on both skill and access (Smith, 2018). - Collective Expertise: The narrative prioritizes team problem-solving and distributed knowledge, often through ensemble blocking and crowded workspaces (Lopez, 2020). - Recognition and Credit: The film dramatizes authorship and attribution, showing how acknowledgement is negotiated in meetings, memos, and mission briefings (Smith, 2018; Lopez, 2020).

Cinematic Techniques The film uses color temperature shifts between home and lab spaces to contrast community and institutional life, while close-ups of documents and chalkboards index the materiality of calculation. Motifs of corridors and thresholds visualize constraint and passage (Lopez, 2020; Green, 2019).

Cultural Context Set against Cold War competition, the film portrays technical excellence as intertwined with civil rights struggles. Archival-inspired design choices ground the narrative in midcentury government work, while dialogue cadence mirrors scientific procedure (Shetterly, 2016; NASA Archives, 2017).

Conclusion By centering mathematicians whose labor is often minimized, the film reorients the heroic frame from singular astronaut to collective computation, making visible the social systems that enable precision (Smith, 2018; Lopez, 2020).

Citation Appendix 1. Shetterly, M. L. (2016). Hidden Figures: The American dream and the untold story of the Black women mathematicians who helped win the space race. William Morrow. 2. Smith, J. (2018). Mathematics and race at midcentury NASA: Labor, expertise, and recognition. *Journal of American Studies*, 52(4), 612–631. <https://doi.org/10.1017/S0021875818000456> (Source parsing failed, unable to retrieve full document content) 3. Lopez, M. (2020). Framing genius: Visual rhetoric in spaceflight cinema. *Cinema Studies Quarterly*, 39(2), 145–169. <https://doi.org/10.1080/00000000.2020.000000> (Source parsing failed, unable to retrieve full document content) 4. NASA Archives. (2017). Langley Research Center photo collection (1958–1969). <https://history.nasa.gov/langley-photos.html> (Source parsing failed, unable to retrieve full document content) 5. Green, T. (2019). Sound and silence in biographical films. *Film Sound Review*, 11(3), 22–38.

Rubric-level Results

#	Rubric	With Skills	Without
0	Substantive Analysis Unaltered	✓	✗
1	Only Validator-Approved Interpretations	✓	✗
2	Strict Adherence to Validated Content	✓	✗
3	Approved Citations Unmodified	✓	✓
4	No Ineligible Film References	✓	✓
5	Accurate Academic Style Application	✓	✗
6	Appropriate Personalized Academic Tone	✓	✓
7	Proper Accessibility Adjustment Implementation	✓	✓
8	Upstream Flag for Structural Inconsistencies	✓	✓
9	Upstream Correction Request for Content Gaps	✓	✓
10	No Independent Structural Error Fixes	✓	✓
11	Issue Flagging via Orchestrator	✓	✓
12	Compliance with Requested Output Format	✓	✓
13	Standardization of Inconsistent Formatting	✓	✗
14	Compliant Heading Hierarchy	✓	✓
15	Standardized Citation Placement	✓	✓
16	Complete Retention of Required Metadata	✓	✓
17	Integrated and Complete Final Document	✓	✓
18	Effective Resolution for Conflicting Constraints	✓	✓
19	Usable and Well-Organized Final Output	✓	✓
20	Focused and High-Value Response	✓	✓
Overall		Solved	Failed

Case Study for Procedural Task Execution (Part 6)

Selected Skills

name: constraint-extraction-and-strict-verification

description: Apply a rigorous process to extract, track, and strictly verify all explicit output constraints (e.g., format, phrasing, limits, artifact citations) prior to responses.

Pre-answer Checklist

- Scan the task and all rubrics for explicit output constraints (format, structure, phrasing, length, citations, forbidden content). - List each constraint as a distinct checklist item. - For every constraint, note its exact wording and origin (task or specific rubric requirement).

Response Procedure

1. Enumerate Constraints: - Parse all task and rubric text for required elements, limits, and forbidden elements. - Build a numbered internal checklist (e.g., "1. Exactly 3 bullet points "2. Cite 'Artifact X' per point "3. No agent names"). 2. Draft in Alignment: - For each response element, reference the checklist and consciously satisfy its associated constraint, marking each as fulfilled. - Mirror required templates, phrasing, or formats exactly—do not paraphrase unless explicitly allowed. - Omit all explicitly forbidden elements (names, advice, extra information, etc.). 3. Link to Sources: - Where required, cross-reference the artifact/source as specified in the checklist at the exact required locations.

Self-verification Steps

- After drafting, review each checklist item: - Confirm every required element is present and matches format/wording/structure precisely. - Check that no extra or forbidden content is included. - Compare the response against the original rubric and task to ensure no constraint is overlooked. - If required to reference specific context or artifacts, verify all are cited correctly. - Only proceed when every checklist item is strictly satisfied.

Common Pitfalls to Avoid

- Missing hidden or implicit constraints embedded in examples or rubrics. - Variations from required phrasing/templates (e.g., paraphrasing when not allowed). - Including agent names, unwarranted explanations, or superfluous recommendations. - Overlooking forbidden structure or the absence of required citations. - Submitting without checking off every constraint from the checklist.

Round 2 Update

name: rubric-to-output-explicit-checklist-mapping

description: Apply an explicit, itemized checklist mapping of rubric requirements to planned outputs, verifying that each requirement is precisely satisfied before finalizing the answer.

Pre-answer Checklist

- Identify all explicit rubric requirements in the task (e.g., evidence references, phrasing, structure, actions to avoid, format, sentence limits). - Break down each rubric requirement into a numbered checklist item.

Response Procedure

1. List all rubric checklist items before drafting the response. 2. For each planned section or statement of your answer, explicitly annotate which rubric item(s) it is intended to satisfy. 3. Draft your response, ensuring each checklist item is addressed: - Use required phrasing and reference named evidence/context as instructed. - Follow mandated organization, structure, and sentence limits. - Do not perform forbidden actions or include prohibited content. - Ensure all required explanations, comparisons, outputs, or formats are included. 4. Clearly link each part of your answer to its corresponding rubric checklist item.

Self-verification Steps

- Systematically cross-check each rubric item against your drafted output: - Confirm all requirements are met literally and unambiguously. - Look for missing, incomplete, or misaligned content/specifications. - Ensure no forbidden actions/content are present. - If any rubric item is not explicitly, fully satisfied, revise the specific output section and re-verify before submitting.

Common Pitfalls to Avoid

- Failing to create or use an explicit rubric checklist before drafting. - Overlooking subtle requirements, such as output format, mandated explanations, or limits. - Assuming a rubric is satisfied without directly locating/cross-referencing it in the output. - Fixing issues in the output without updating or re-verifying the checklist. - Neglecting to reference required context/evidence, or using incorrect phrasing.

Case Study for Procedural Task Execution (Part 7)

Selected Skills(Continue)

Round 3 Update

name: constraint-trace-to-pre-answer-validation

description: "Extract all explicit constraints and rubric requirements, construct a pre-answer checklist, and systematically validate every item against the drafted output before final submission, flagging any ambiguities or missing information."

Pre-answer checklist

1. Scan the entire conversation context (instructions, rubrics, artifacts). 2. List every explicit output constraint, including: - Required sections/headings and exact naming - Output format (e.g., bullet list, YAML, code block) - Word/line/character limits - Data/citation requirements - Justification or rationale mandates - Ambiguity handling directives (e.g., flag uncertainties) - Prohibited content or conflict exclusions - Other stated requirements 3. Record each constraint as a clear checklist item. 4. Flag and list any ambiguous, under-specified, or conflicting requirements.

Response procedure

1. Build your response incrementally, addressing each checklist item in sequence. 2. For every checklist item, explicitly create the corresponding output element (e.g., section, format, justification). 3. When encountering ambiguity or underspecified instructions, note the gap in the draft and prepare a flag in the output per the checklist.

Self-verification steps

1. Before submitting: - For each checklist item, locate and review the exact corresponding element in the draft response. - Mechanically check for: - Exact section headings/names - Strict adherence to format and limits (count words/lines/characters if required) - Inclusion of evidence/justification where specified - Flagging of uncertainties or ambiguities - Absence of prohibited content or scope violations 2. If an item is missing, misapplied, or ambiguous, revise or explicitly flag the gap in the output. 3. Only submit after every checklist item is validated or flagged accordingly.

Common pitfalls to avoid

- Missing or paraphrasing required section/heading names - Inventing unstated rubric criteria - Ignoring or loosely applying format or count restrictions - Failing to include mandated justification or rationale - Overlooking ambiguity flagging requirements - Presenting outputs with unvalidated or unaddressed checklist items

Discussion

The skill forces a pre-answer checklist that catches subtle but critical violations. In the College Movie Report case, the `without_skill_response` appended a "Formatting notes:" block outside the requested Markdown report, adding unauthorized commentary and breaking the "single, polished Markdown report" constraint. This extraneous text directly caused failures on Rubric 0 (modifying intellectual substance by adding unrequested notes), Rubric 1 (suggesting interpretations not approved – the notes imply the agent's own layout reasoning), Rubric 2 (including non-validated material), Rubric 5 (the notes disrupted APA-7th presentation), and Rubric 13 (output format not a clean Markdown report). In contrast, the `ours_response` delivered only the flagged report, with no extra text. The skill's self-verification step explicitly checks "no extra or forbidden content" and "strict adherence to format," which would have rejected the notes block. Thus, the skill transforms a functionally correct answer into a strictly compliant one, closing gaps that otherwise undermine academic integrity and user expectations.

Figure 7: Case study for Procedural Task Execution.

Case Study for Empirical Discovery & Simulation (Part 1)

Task Information

- **UID:** 1c4e93a4-ccb0-42c9-bb95-c14fbaad50c1
- **Model:** GPT-4.1
- **Task:** Produce the required four-section output. Requirements:1) Establish seat 1 as the timing reference and compute per-seat catch/finish offsets and per-seat session averages with standard deviations using the provided stroke excerpts. Identify all seats exceeding $\pm 35\text{ms}$ average or $>35\text{ms}$ variability and classify timing status and consistency per definitions.2) Compute the overall crew sync score using the specified formula, bound it to 0–100, and classify overall quality. Provide a clear, concise explanation.3) Analyze pairs (port/starboard: 1–2, 3–4, 5–6, 7–8; sequential pairs: 1–2, 2–3, ..., 7–8) for washing-out or compounding patterns. Call out the bow and stern pairs explicitly.4) Perform rhythm analysis per minute using the rhythm consistency formula and drive:recovery ratio. Classify rhythm quality and label ratios as optimal/acceptable/rushed/dragged.5) Identify any unusual global patterns (e.g., the disruption around 07:48) and handle appropriately. Do not alter or smooth clean timing data.6) Quantify performance impact: estimated efficiency loss if overall sync score <90 , 2k time impact using 6:00 (360s) baseline, and wasted power using 1710 W.7) Include precise priorities for Insights (what to address first) and explicit visualization requests for the Dashboard (timeline, heatmap), with key callouts. Output format (all sections required):<analysis>[Concise reasoning on data handling, calculations performed, anomaly treatment, and verification against artifacts]</analysis><synchronization_report>[Crew Sync Score and classification; Estimated Efficiency Loss; Problem Seats (with averages and consistency); Problem Pairs; Rhythm Analysis (consistency, drive:recovery, stroke rate variability); Temporal Trends; Impact (time loss and wasted power)]</synchronization_report><to_insights_agent>[Prioritized list of issues and suggested focus areas for interventions; include unusual pattern escalation]</to_insights_agent><to_dashboard_agent>[Visualization requests and key callouts for rendering]</to_dashboard_agent>

System Prompt

Agent 4: Synchronization Agent - Rowing Telemetry Multi-Agent System Overall System Context: The 9-Agent Rowing Performance Framework You are operating within a 9-agent system designed to transform complex rowing telemetry data from Peach Innovations' PowerLine system into actionable coaching insights for crew performance optimization. The Objective: To analyze stroke-by-stroke telemetry data (force curves, oar angles, boat speed, acceleration) and provide coaches with clear, prioritized recommendations for improving power output, technique, crew synchronization, and overall racing performance. The Workflow: The system operates in a coordinated workflow involving the following agents:

A1: Orchestrator Agent - coordinates all operations, interprets user requests, manages workflow A2: Data Ingestion Agent - imports and cleans raw PowerLine CSV telemetry files A3: Context/Environmental Agent - normalizes data based on weather, water conditions, boat configuration A4: Synchronization Agent - You - analyzes crew timing and coordination across all seats A5: Biomechanics Agent - evaluates rowing technique and identifies inefficiencies A6: Output/Power Agent - calculates power output, boat speed, and efficiency metrics ... (remaining response omitted)

Agent 4 Prompt: The Synchronization Agent (Crew Timing Specialist) You are the Synchronization Agent, the crew timing specialist who analyzes how well rowers coordinate their movements. Perfect synchronization means all rowers catch and finish simultaneously, creating smooth boat run and maximum efficiency. Poor synchronization creates rhythm disruptions, reduces power transfer, and slows the boat. Your ultimate goal is to identify timing issues, quantify their impact on performance, and flag specific seats or pairs that need timing corrections.

Inputs

<from_data_ingestion> STROKE_TIMING_DATA (Catch/finish timestamps for all seats, stroke-by-stroke)
</from_data_ingestion>

<from_context_agent> NORMALIZED_DATASET (Cleaned data with environmental adjustments)
</from_context_agent>

<from_shared_context> HISTORICAL_SYNC_DATA (Previous sessions' synchronization metrics for trend analysis) </from_shared_context>

Your Role: Crew Timing Analysis & Rhythm Assessment You must calculate timing offsets, quantify synchronization quality, identify problem seats, analyze rhythm consistency, and generate sync trends over the session.

Core Functions: TIMING OFFSET CALCULATION: Measure catch/finish timing relative to stroke seat SYNC SCORE COMPUTATION: Generate 0-100% crew synchronization rating PROBLEM IDENTIFICATION: Flag seats that are early/late and by how much RHYTHM ANALYSIS: Assess stroke-to-stroke timing consistency

PAIR PROBLEM DETECTION: Identify specific seat pairs with timing conflicts TEMPORAL TRENDS: Track how sync changes over session duration IMPACT QUANTIFICATION: Estimate efficiency loss due to poor sync Chain-of-Thought Synchronization Process

Phase 1: Timing Reference Establishment
Set up timing baseline: Identify Stroke Seat as Reference: Seat 1 (stroke seat) is always the timing reference, All other seats measured relative to stroke seat, Stroke seat's catch time = 0ms offset (by definition) Extract Catch/Finish Times Per Stroke:

Case Study for Empirical Discovery & Simulation (Part 2)

System Prompt(Continue)

For each stroke across all seats:

```
{
  "stroke_number": 1,
  "timestamp": 1699545601234,
  "seat_timings": [
    {
      "seat": 1,
      "catch_time": 0.000,
      "finish_time": 0.750,
      "recovery_time": 1.100
    },
    {
      "seat": 2,
      "catch_time": 0.015,
      "finish_time": 0.768
    },
    ...
  ]
}
```

Calculate Absolute Timing Offsets: For each seat X: $\text{catch_offset}[X] = \text{seat}[X].\text{catch_time} - \text{seat}[1].\text{catch_time}$, $\text{finish_offset}[X] = \text{seat}[X].\text{finish_time} - \text{seat}[1].\text{finish_time}$ Positive = late (after stroke seat), Negative = early (before stroke seat)

Phase 2: Per-Seat Offset Analysis

Analyze each rower's timing: Calculate Session-Average Offsets: For each seat: $\text{avg_catch_offset} = \text{mean}(\text{catch_offset across all strokes})$, $\text{std_dev_catch} = \text{standard_deviation}(\text{catch_offset})$, $\text{avg_finish_offset} = \text{mean}(\text{finish_offset})$, $\text{std_dev_finish} = \text{standard_deviation}(\text{finish_offset})$

Classify Timing Status: Based on average catch offset: On Time: -10ms to +10ms (excellent), Slightly Late: +10ms to +35ms (acceptable), Moderately Late: +35ms to +50ms (needs attention), Very Late: >+50ms (serious problem), Slightly Early: -10ms to -35ms (acceptable), Moderately Early: -35ms to -50ms (needs attention), Very Early: <-50ms (serious problem)

Assess Consistency: Based on standard deviation: Very Consistent: $\text{std_dev} < 15\text{ms}$, Consistent: $\text{std_dev} 15\text{-}25\text{ms}$, Inconsistent: $\text{std_dev} 25\text{-}40\text{ms}$, Very Inconsistent: $\text{std_dev} > 40\text{ms}$ Note: A seat can be consistently off-time (e.g., always 30ms late) or inconsistently on-time (e.g., varies -10ms to +10ms)

Generate Per-Seat Report:

```
{
  "seat": 3,
  "avg_catch_offset_ms": -35,
  "catch_offset_std_dev": 18,
  "avg_finish_offset_ms": -28,
  "finish_offset_std_dev": 15,
  "sync_status": "moderately_early",
  "consistency": "very_consistent",
  "interpretation":
    "Seat 3 is consistently catching 35ms early—a clear timing issue,
    but at least consistent about it."
}
```

Phase 3: Crew Synchronization Score Calculation

Compute overall sync quality: Calculate Crew Sync Score (0-100): Components: $\text{avg_absolute_offset}$: Average of $|\text{catch_offset}|$ across all seats (excluding seat 1), avg_std_dev : Average of std_dev across all seats, num_outliers : Count of seats with $|\text{offset}| > 50\text{ms}$ $\text{sync_score} = 100 - (\text{avg_absolute_offset} \times 0.5) - (\text{avg_std_dev} \times 0.3) - (\text{num_outliers} \times 5)$, Bounds: $\max(0, \min(100, \text{sync_score}))$ Classify Sync Quality:

Excellent: $\text{sync_score} > 95$ (all seats within $\pm 20\text{ms}$, very consistent), Good: $\text{sync_score} 85\text{-}95$ (all seats within $\pm 35\text{ms}$, mostly consistent), Fair: $\text{sync_score} 70\text{-}85$ (some seats $\pm 35\text{-}50\text{ms}$, moderate consistency), Poor: $\text{sync_score} < 70$ (multiple outliers or very inconsistent)

Case Study for Empirical Discovery & Simulation (Part 3)

Context

Turn 1 - 2025-11-12 T08:05:12 Message from: Orchestrator (A1):Context: Morning practice session 2025-11-12_morning_practice” with the first varsity 8+. You will own crew timing and synchronization analysis. Deliver a structured report for the coach and feed-throughs to the Insights and Dashboard agents. Target delivery within the hour.

Turn 2 - 2025-11-12 T08:07:01 Message from: Data Ingestion (A2):I’ve imported and cleaned the Peach PowerLine CSVs. Timestamps are aligned to UTC and normalized to the session clock. Sensor health checks pass. Note: small jitter flags existed on seat 1 during strokes in the 07:48:20–07:48:40 window, but cross-checking with accelerometer and angle streams indicates valid readings (not a hardware fault). Artifacts 2 and 3 provide representative stroke timing excerpts.

Turn 3 - 2025-11-12 T08:08:37 Message from: Context Agent (A3):Environmental normalization applied. Crosswind increased 10% after 07:47; chop rose slightly. No correction applied to timing offsets, only to speed and acceleration features (see Artifact 5). Timing data should be analyzed as-is.

Turn 4 - 2025-11-12 T08:10:22 Message from: Coach (User):Subjective feel: bow pair looked out of rhythm late, and there was a weird moment around 07:48 when everything fell apart for 20–30 seconds. Please confirm what happened, quantify how much it cost us over a 2k, and identify which seats/pairs to prioritize.

... (remaining response omitted)

Artifact 1 — Lineup & Seat Mapping (Seats 1–8, with sides) Seat 1: Stroke (Starboard) Seat 2: Port Seat 3: Starboard Seat 4: Port Seat 5: Starboard Seat 6: Port Seat 7: Starboard Seat 8: Bow (Port) Note: Seat 1 (stroke) is the reference for timing. Artifact 2 — Stroke Timing Excerpts (Window A: 07:40–07:45)

Format per stroke:

```
{
  stroke\_number': <int>,
  timestamp': HH:MM:SS.mmm',
  seat\_timings': [
    {
      seat': n,
      catch\_time': seconds,
      finish\_time': seconds
    }, ... ]
}
```

Assume seat 1 catch_time = 0.000 by definition per stroke. Data (selected 12 strokes):

```
{
  stroke\_number: 121,
  timestamp: 07:41:12.100,
  seat\_timings:
  [
    {seat:1,
     catch\_time:0.000,
     finish\_time:0.740} ,
    {seat:2,
     catch\_time:0.018,
     finish\_time:0.759
    },...
  ]
  stroke\_number: 122,
  timestamp: 07:41:13.460,
}

{
  seat\_timings:
  [
    {seat:1,
     catch\_time:0.000,
     finish\_time:0.742} ,
    {seat:2,
     catch\_time:0.019,finish\_time:0.762},
    ..
  ],
  stroke\_number: 123,
  timestamp: 07:41:14.820,
}
```

... (remaining response omitted)

Case Study for Empirical Discovery & Simulation (Part 4)

Rubrics

0. Does the model refuse to override timing data even if it seems wrong? Fail if the model overrides clean timing data rather than reporting it accurately
1. Does the model escalate unusual patterns (e.g., entire crew suddenly loses sync) to Insights Agent? Fail if unusual sync patterns are not escalated to Insights Agent
2. Does the model escalate all identified sync problems to Insights Agent for drill recommendations? Fail if sync problems are not escalated to Insights Agent
3. Does the model identify Seat 1 (stroke seat) as the timing reference? Fail if stroke seat is not used as the timing reference or if reference is incorrect
4. Does the model correctly calculate timing offsets relative to stroke seat? Fail if timing offset calculation formula is incorrect or sign convention is wrong
5. Does the model correctly calculate sync score using the specified formula? Fail if sync score formula is incorrect or weights are wrong
6. Does the model bound sync score between 0 and 100? Fail if sync score is not bounded to 0-100 range
7. Does the model correctly calculate rhythm consistency? Fail if rhythm consistency formula is incorrect
8. Does the model correctly calculate efficiency loss for $\text{sync_score} < 90$? Fail if efficiency loss formula is incorrect or threshold is wrong
9. Does the model calculate time impact correctly for racing scenarios? Fail if time impact calculation is incorrect
10. Does the model produce all four required output sections? Fail if any required output section is missing
11. Does the model analyze both port vs starboard pairs and sequential pairs? Fail if both pair types are not analyzed"

Response with Ctx2Skill Skills (Solved ✓)

<analysis>Stroke timing data from Artifacts 2 (07:41 steady state) and 3 (07:50 post-anomaly) were analyzed using seat 1 (Stroke) as the timing reference (per Artifact 1 and explicit instructions). For each window, per-seat catch and finish offsets (in seconds) were extracted for all available strokes. Session means and standard deviations (stddev) were computed per seat; all calculations were based on raw (non-rounded) numbers per rubric. Offsets exceeding $\pm 35\text{ms}$ average or $>35\text{ms}$ stddev were highlighted, and outlier seats/pairs were flagged precisely. Only clean, verified data from Artifacts 2–3 were used; preliminary numbers in Artifact 6 were discarded due to methodology flaws (wrong reference, sign errors). Overall crew sync score was computed with the rubric formula: $\text{sync_score} = 100 - (\text{avg_abs_offset} \times 0.5) - (\text{avg_std_dev} \times 0.3) - (\text{num_outliers} \times 5)$, with values for offset and stddev in milliseconds and proper outlier counting. Score bounds were strictly enforced. Port/starboard (1–2, 3–4, etc.) and sequential seat pairs were reviewed for timing directionality (washing out/compounding), with explicit checks for bow pair (7–8) and stern pair (1–2). Rhythm analysis covered stroke rate variability and drive/recovery using Artifact 7; rhythm consistency was calculated per minute and ratios classified by spec. The anomalous region at ~07:48 (coach/cox report, Artifact 5) was treated as a disruption, but no smoothing was done. Any performance impact (efficiency, 2k time, wasted power) was calculated only if sync score < 90 , per rubric and using 1710 W and 6:00 baseline (Artifact 7 and A6 input). Il priorities and visualization requests align with the most severe and actionable findings, and are annotated per the guidelines. Synchronization Report: Crew Sync Score: 85% (Fair); Estimated Efficiency Loss: 1.5%. Problem Seats: Seat 3: -39ms early, stddev 3ms, timing status: moderately early, consistency: very consistent; Seat 7: +56ms late, stddev 6ms, timing status: very late, consistency: very consistent; Seat 2: +32ms late, stddev 2ms, timing status: slightly late, consistency: very consistent; All other seats within $\pm 20\text{ms}$ average; std devs all $< 15\text{ms}$.

Problem Pairs: Seats 3 (starboard), 4 (port): Washing out — seat 3 consistently early, seat 4 nearly on time (diff: ~47ms), compounding timing gap, severity: moderate; Seats 7 (starboard), 8 (bow, port): Bow pair out of sync — seat 7 very late (+56ms), seat 8 slightly late (+17ms), bow rhythm suffers, severity: serious; Stern pair (seats 1–2): Seat 2 slightly late vs stroke (diff: 32ms), minor compounding effect. Rhythm Analysis: Consistency per minute all rated Very steady across 07:41–07:53; Drive-to-Recovery ratios 0.36–0.40 (Optimal throughout); Stroke rate variability 0.35–0.74 spm (low and very steady); overall rhythm remains excellent.

Temporal Trends: Pre-disruption sync 89–91%, 07:48 disruption drops score to low 80s, post-disruption stabilizes at 84–85%; trend shows clear sync degradation sustained after the 07:48 launch incident, with bow pair mismatch persisting. Impact: Estimated 2k race time loss ~5.4 seconds; wasted crew power ~26W.

To Insights Agent: Priority Issues for Recommendations: 1. Bow pair (seats 7 & 8): Seat 7 consistently very late (+56ms), driving sustained bow rhythm breakdown post-disruption (highest priority). 2. Seat 3: persistently moderately early (-39ms), creating measurable pair mismatch with seat 4. 3. Minor: Seat 2 slight lateness with limited overall impact. Suggested Focus: Bow pair timing reset and post-interruption recovery drills; targeted individual timing correction for seat 3; embed post-anomaly reset strategies into crew workflow. To Dashboard Agent: Visualization Requests: per-seat minute-by-minute timing offset timeline, minute-level sync score trend plot, seat-by-time catch offset heatmap, annotated markers for the 07:48 disruption and post-disruption drift. Key Callouts: highlight seat 7 and seat 3 as primary outliers; label sustained post-07:48 sync drop and persistent bow pair timing divergence; flag bow pair heatmap row for chronic latency and mismatch.

Case Study for Empirical Discovery & Simulation (Part 6)

Response without Skills (Failed X)

Reasoning and Methodology: Seat 1 (stroke) is correctly used as the timing reference per definition (Artifact 1, Orchestrator instructions). Offsets for all seats are computed as the difference in catch and finish times to seat 1 per stroke, without any rounding or smoothing—even for reporting or dashboard purposes—as required for accuracy and intervention actionability. **Data Handling:** Selected representative strokes from both early (Window A, 07:40–07:45) and post-anomaly (Window B, 07:50–07:55) periods. For per-seat average and std dev, both windows are included for a wider snapshot and to properly reflect rhythm/anomaly-induced variability.

Offset Calculation: For each seat, computed mean catch offset (in ms, i.e. seconds (times) 1000) and standard deviation across all strokes in given windows. Early catch: negative; Late catch: positive. Per-agent definitions for on time,” slightly/moderately/very early/late,” and consistency classifications applied as specified.

Sync Score: Implemented prescribed sync score formula using calculated session averages and variability. Outliers identified per definitions.

Problem Pair & Rhythm Analyses: Port/starboard and sequential pairs checked for >35ms mean offset and for seats exhibiting “split” (one early, one late) patterns. Bow pair (7–8) and stern (1–2) explicitly analyzed.

Rhythm Calculation: Used Artifact 7 to extract drive/recovery and per-minute stroke rate averages/STDs; rhythm consistency classified as per formula and ratio thresholds.

Disruptions: As directed, noted and isolated anomaly region (07:48) in the narrative; refrained from timing correction, but flagged it for escalated attention.

Impact Quantification: Based on calculated sync score, computed efficiency loss (if <90), 2k time cost (over 360s), and wasted power (from 1710 W).

Results Verified: Crosschecked junior analyst’s numbers (Artifact 6) and found sign and reference errors: our calculation reverses polarity, improving accuracy and diagnostic value.

Synchronization Report: Crew Sync Score: 85% (Fair); Estimated Efficiency Loss: 1.5%.

Problem Seats: Seat 3: –36ms early average, std dev 4ms (moderately_early, very_consistent); Seat 7: +54ms late average, std dev 6ms (very_late, very_consistent); Seat 2: +30ms late average, std dev 7ms (slightly_late, very_consistent); Seat 4: +10ms late average, std dev 2ms (on_time, very_consistent); Seat 5: +15ms late average, std dev 2ms (on_time/slightly_late, very_consistent); Seat 6: +8ms late average, std dev 2ms (on_time, very_consistent); Seat 8: +13ms late average, std dev 3ms (on_time/slightly_late, very_consistent).

Problem Pairs: Stern Pair (1–2): Seat 2 consistently 30ms late relative to stroke; Three–Four Pair (3–4): Seat 3 substantially early, Seat 4 just on time, producing 46ms delta with washing-out risk; Five–Six Pair (5–6): Mild 15ms vs 8ms late mismatch, acceptable; Bow Pair (7–8): Seat 7 severely late (+54ms), seat 8 slightly late (+13ms), 41ms split causing bow rhythm disruption; Sequential Pair 6–7: Abrupt timing jump from +8ms to +54ms, confirming coach observation of bow out-of-rhythm.

Rhythm Analysis: Overall rhythm consistency 0.98 (Excellent, steady rhythm); Drive-to-Recovery Ratio 0.36–0.39 (Optimal); Stroke rate variability 0.35–0.74 spm, exceptionally steady; despite individual seat offset errors, crew overall pacing remains stable with no rushed or dragged sequencing.

Temporal Trends: Pre-anomaly sync score 88–90%; post 07:48 launch disruption drops to 83–85%, partial recovery without returning to peak; the incident caused acute bow pair desync rather than systemic technical failure, matching coach and Artifact 5 contextual reports.

Impact: 1.5% efficiency loss yields 5.4 seconds slower over 2000m baseline; wasted crew power calculated at 25.7W; correcting seat 7 lateness and seat 3 early timing can recover 5+ seconds in 2k racing and reclaim 26W of usable power.

To Insights Agent: Priority Issues: 1. Highest priority: correct seat 7 persistent +54ms late timing and rebalance bow pair 7–8, the main source of post-disruption desync. 2. Second priority: fix seat 3 systematic –36ms early catches, a recurring historical trend risking rhythm tearing. 3. Tertiary: monitor mild seat 2 lateness with no immediate urgent action.

Suggested Focus: Deliver cox/coach guidance on targeted bow pair catch synchronization post-interruption; train seat 3 to match stroke seat catch timing; integrate post-disruption reset drills for bow and mid-boat sequencing; further evaluate crew susceptibility to external lane disruption events.

To Dashboard Agent: Visualization Requests: seat-by-seat catch offset timeline 07:40–07:55; per-minute sync score trend with vertical annotation at 07:48:28 launch event; seat-by-time heatmap color-coded for early/on-time/late offsets; explicit overlays and annotations for bow pair 7–8 and sequential 6–7 timing divergence.

Key Callouts: Label seat 7 as primary persistent late outlier; flag seat 3 recurring early timing habit; mark permanent partial sync degradation after the 07:48 lane crossing incident.

Case Study for Empirical Discovery & Simulation (Part 6)

Rubric-level Results

#	Rubric	With Skills	Without
0	Refuse Overriding Raw Timing Data	✓	✓
1	Escalate Unusual Sync Patterns	✓	✓
2	Escalate All Sync Problems	✓	✗
3	Correct Stroke Seat Reference	✓	✓
4	Accurate Timing Offset Calculation	✓	✓
5	Compliant Sync Score Formula	✓	✗
6	Sync Score Range Bounding	✓	✓
7	Correct Rhythm Consistency Calculation	✓	✗
8	Compliant Efficiency Loss Calculation	✓	✗
9	Accurate Racing Time Impact Calculation	✓	✓
10	Complete Required Output Sections	✓	✓
11	Analyze Both Pair Types	✓	✓
Overall		Solved	Failed

Selected Skills

name: structured-output-self-verification

description: Systematically verify that answers comply with all rubrics and template constraints, ensuring precise format, content coverage, and rubric-by-rubric adherence prior to submission.

Pre-answer checklist

- Gather all rubrics, instructions, and explicit constraints from the task prompt. - Extract all required formats (e.g., tables, bullet points), template structures, and section ordering. - List every explicit content requirement (e.g., required phrases, analytic statements, event time references, escalation notes).

Response procedure

1. Draft the answer according to the extracted rubrics, constraints, and templates. 2. Explicitly label or mark each required section and content piece while composing. 3. Apply required templates (e.g., tables, section headings) exactly as specified, not improvising formats.

Self-verification steps

1. Rubric-by-rubric check: - For each rubric item or constraint, locate and review the corresponding output element. - Confirm that the output fulfills the exact requirement (not just approximately). - No required rubric element should be missing or only loosely satisfied. 2. Template and format validation: - Ensure section order, heading style, table structure, and list formatting strictly follow instructions. - Validate any limits (e.g., number of bullets, maximum columns, forbidden formatting like extra narrative text or stray Markdown). 3. Explicit content checks: - Confirm all mandated phrases, statements (analytic or factual), references, times, and notes are present and exactly worded if required. 4. Limit enforcement: - Check for compliance with all word, content, or presentation limits set by the prompt. 5. Revise the answer if any non-compliant element is found, and repeat the above steps until fully satisfied. 6. Remove any self-verification notes or meta-comments before final submission.

Common pitfalls to avoid

- Adding extra narrative, filler text, or explanations not required by the prompt. - Using allowed structures or formats incorrectly (e.g., unordered instead of ordered lists, wrong table borders/headings). - Missing required analytic elements or precise wording. - Exceeding or not meeting explicitly stated content/word/section limits. - Overlooking hidden or implicit constraints in the prompt—verify every instruction line.

Discussion

The skill transforms a mathematically plausible but formula-violating answer into one that strictly complies with every analytic rubric. In the rowing telemetry case, the without_skill_response miscalculated the sync score (used a different formula, failing rubric 5), omitted the required rhythm consistency calculation (rubric 7), and incorrectly computed efficiency loss (used $(90-85) \times 0.3\%$ instead of the official $(90 - \text{sync_score}) \times 0.3\%$ when $\text{sync_score} < 90$ – rubric 8). It also failed to escalate all identified sync problems to the Insights Agent (rubric 2) by only listing symptoms without explicitly flagging them as priorities for drill recommendations. In contrast, the ours_response followed the skill's pre-answer checklist: it extracted the exact formula $\text{sync_score} = 100 - (\text{avg_abs_offset} \times 0.5) - (\text{avg_std_dev} \times 0.3) - (\text{num_outliers} \times 5)$ and computed 85% correctly; it applied the rhythm consistency formula $1 - (\text{stroke_rate_std} / \text{stroke_rate_mean})$ to each minute; it used $(90 - \text{sync_score}) \times 0.3\%$ for efficiency loss; and it dedicated a full <to_insights_agent> block with "Priority Issues for Recommendations" and "Suggested Focus," explicitly escalating every problem seat and pair. The skill's rubric-by-rubric self-verification step catches these precise deviations, turning an otherwise competent analysis into a fully compliant, coach-actionable report.

Figure 8: Case Study for Empirical Discovery & Simulation.

Prompt Prompt Used for Challenger

You are an expert specializing in creating evaluation tasks for language models. You will be given a multi-turn conversation context and a required number of tasks. Your job is to generate $\{M\}$ new evaluation tasks, each with its own rubrics.

Task Design Rules

1. **Context-grounded:** The task MUST require information from the conversation context to answer correctly. A model that has not read the context should be unable to answer well.
2. **Complexity:** Include the following complexity factors:
 - Reference to specific facts, entities, or data in the context
 - Output format requirements (e.g., structured sections, tables, numbered lists, slides)
 - Exact numerical constraints (e.g., "exactly 3 examples", "no more than 200 words")
 - Multi-step reasoning or multi-part deliverables (e.g., "first analyze X, then propose Y")
 - Compliance with behavioral rules set in the system prompt (if present)
3. **Phrasing:** The task should read as a natural user request. It can be a question, an instruction, or a conversational follow-up — avoid overly formal or artificial phrasing.
4. **Non-trivial:** The task should require genuine reasoning, synthesis, or careful reading — not a simple lookup or yes/no question.
5. **Diversity:** Each task in the batch must target a DIFFERENT aspect of the conversation context. Avoid generating variations of the same question. Vary the complexity factors, rubric categories, and reasoning demands across tasks.

Rubric Design Rules

Generate the corresponding rubrics. Each rubric is a single sentence defining one specific, binary (pass/fail) criterion for judging a model's response.

1. **Type balance** — include rubrics from multiple categories:
 - Content inclusion: "The response should [include/mention/identify] [specific content]."
 - Content exclusion: "The response should not [include/mention/do] [specific thing]."
 - Format/structure: "The response should [format requirement]."
 - Accuracy: "The response should correctly [state/calculate/identify] [specific fact]."
 - Constraint compliance: "The response should [meet exact constraint]."
 - Remaining: sequence/ordering, tone/style, or domain-specific logic as appropriate.
2. **Binary and verifiable:** Each rubric must be unambiguously checkable as pass or fail. Avoid vague criteria like "the response should be good" or "the response should be comprehensive".
3. **Specificity:** Name exact entities, numbers, facts, or sections from the context. Bad: "The response should mention relevant data." Good: "The response should state that the growth rate was 12.5% in Q3."
4. **Illustrative examples:** For ~20% of rubrics, append "For example, ..." to clarify what satisfies the criterion.
5. **Independence:** Each rubric tests one distinct aspect. Do not bundle multiple checks into a single rubric.
6. **System prompt awareness:** If the context includes a system prompt with behavioral rules (persona, tone, formatting rules, forbidden content), include 2-3 rubrics that verify compliance with those rules — even if the task does not explicitly mention them.

Output Format

Output ONLY valid JSON:

```
{
  "tasks": [
    {
      "task": "task 1 content as a user message",
      "rubrics": ["rubric 1", "rubric 2", ...]
    },
    {
      "task": "task 2 content as a user message",
      "rubrics": ["rubric 1", "rubric 2", ...]
    }
  ]
}
```

Available Skills (if any)

You have access to the following specialized skills. When a task matches a skill's description, follow its instructions.

$\{\text{The generated skills for challenger (if any)}\}$

Conversation Context

$\{\text{context}\}$

Your Task

Based on the conversation context above, generate exactly $\{M\}$ evaluation task(s), each with its own rubrics, following the rules in your instructions. Each task should test a DIFFERENT aspect of the context. Output ONLY the JSON object.

Figure 9: **Prompt used for the Challenger.** The Challenger receives the context and generates M tasks, each with binary rubrics, following the task and rubric design rules specified in this prompt.

Prompt Used for Challenger Proposer

You are an expert analyst specializing in adversarial benchmark design. Your job is to analyze why some of the Challenger's tasks were too easy for the Reasoner, and propose a skill to improve the Challenger's task and rubric generation ability.

Context

The Challenger generates a batch of tasks, each with its own rubrics, for a given conversation context. Each task is scored independently (0/1) by a Judge. You are given the tasks that the Reasoner passed (score = 1), meaning those tasks were too easy or their rubrics were too lenient. Both sides improve in parallel: the Challenger improves from passed tasks, while the Reasoner improves from failed tasks.

Analysis Process

Before proposing a skill, work through these steps:

Step 1: Failure Diagnosis

- Were the tasks too similar to each other, failing to test diverse aspects of the context?
- Were the rubrics too lenient, vague, or easy to satisfy by chance?
- Were the tasks too straightforward (simple lookup, yes/no, surface-level questions)?
- Did the rubrics fail to test deep understanding of the context?
- Were there too few complexity factors (format, constraints, multi-step reasoning)?
- Could the tasks be answered without carefully reading the full context?
- Did the rubrics lack specificity (no exact entities, numbers, or facts from context)?
- Was there insufficient variety in complexity factors across tasks?

Step 2: Existing Skill Check

- Review the Challenger's existing skills listed in the query
- Does any existing skill already cover this weakness?
- If yes, why did it fail? Should it be EDITED rather than replaced?

Step 3: Pattern Identification

- Is this a recurring failure pattern (e.g., always generating single-step tasks)?
- What class of improvement would have the broadest impact across a batch of tasks?

Skill Design Rules

1. The skill should teach the Challenger concrete strategies for generating harder, more discriminative, and more diverse batches of tasks with tighter rubrics.
2. Include actionable checklists, patterns, or templates — not generic advice.
3. Build on (don't repeat) any existing skills the Challenger already has.
4. The skill should generalize beyond this single failure case.
5. Focus on task design techniques: multi-step reasoning, cross-reference requirements, format constraints, context-dependent facts, implicit system prompt compliance, and diversity across tasks in a batch.

Anti-Patterns to Avoid

- DON'T propose a new skill if an existing one covers similar ground — propose an EDIT instead.
- DON'T create narrow skills that only fix one specific context — ensure broad applicability.
- DON'T propose vague improvements like "make tasks harder" — specify HOW.

Output Format

Output ONLY valid JSON:

```
{
  "action": "create or edit",
  "target_skill": null or "existing-skill-name",
  "analysis": "failure analysis",
  "skill_name": "short-kebab-case-name",
  "skill_description": "when to apply this skill",
  "proposed_skill": "High-level description of what the skill should do",
  "justification": "Why this skill addresses the identified gap, referencing specific failure evidence"
}
```

Round {i} Failure Analysis

Conversation Context

{context}

All Tasks Overview

{solutions and outcomes from reasoner and judge}

Analysis Focus

The Challenger generated {M} tasks. The Reasoner PASSED {passed_num} of them, meaning those tasks were too easy. Analyze the PASSED tasks below to identify why they failed to challenge the Reasoner. Look for patterns: were rubrics too lenient? Were tasks too straightforward? Were there not enough complexity factors? Was there insufficient diversity across tasks?

Detailed Traces for Analysis

{traces information for each passed task, including the task content, rubrics, reasoner's response, judge's evaluation, per-rubrics status}

Existing Skills for Challenger

{The generated skills for challenger (if any)}

Analyze the failure pattern and propose a skill improvement. Output ONLY the JSON object.

Figure 10: **Prompt used for the Challenger Proposer.** The Challenger Proposer analyzes tasks that the Reasoner passed (i.e., tasks that were too easy) and proposes a new or edited skill to improve the Challenger's task generation ability.

Prompt Used for Challenger Generator

You are an expert skill developer. Your job is to implement a concrete, actionable SKILL.md for a Challenger agent based on a proposal from the Proposer.

Context

The Challenger agent generates batches of evaluation tasks (each with its own rubrics) for conversation contexts. The skill you create will be injected into the Challenger's system prompt to improve its ability to generate diverse, challenging batches of tasks.

Implementation Rules

1. **Actionable, not abstract:** The skill must contain concrete strategies, checklists, patterns, or templates that the Challenger can directly follow when generating tasks.
2. **Concise:** Every token in the skill competes with the conversation context for the model's attention. Challenge each sentence: "Does this add actionable value?" Remove filler.
3. **Structured:** Use clear markdown sections:
 - What to do (specific steps or checklist)
 - Examples of good vs bad patterns (brief, illustrative)
 - Common pitfalls to avoid
4. **YAML frontmatter:** The SKILL.md MUST start with a YAML frontmatter block containing ONLY these two fields:

```
—
skill_name: short-kebab-case-name
skill_description: One-sentence description of when to apply this skill
—
```

Do NOT include any other fields (no title, name, description, tags, or any other keys).

5. **Complementary:** If there are existing skills, the new skill should complement (not overlap with) them. Reference existing skills where relevant.
6. **Build on the proposal:** The Proposer has analyzed the failure and described what the skill should do. Your job is to turn that high-level description into a well-structured SKILL.md.

Output Format

Output ONLY valid JSON:

```
{
  "skill_content": "The complete SKILL.md content (markdown string with YAML frontmatter)",
  "reasoning": "Brief explanation of key implementation decisions"
}
```

Skill Proposal

{skill proposal from challenger proposer}

Existing Skills

{existing challenger skills}

Implement the skill as a complete SKILL.md. Output ONLY the JSON object.

Figure 11: **Prompt used for the Challenger Generator.** The Challenger Generator implements the Proposer's skill specification into a concrete, actionable SKILL.md that will be injected into the Challenger's system prompt.

Prompt Used for Reasoner Proposer

You are an expert analyst specializing in language model reasoning evaluation. Your job is to analyze why a Reasoner agent failed to satisfy task rubrics on specific tasks, and propose a skill to improve its problem-solving ability.

Context

The Reasoner receives a conversation context + a batch of tasks (each with its own rubrics) and must produce a response for each task satisfying all its rubrics. Each task is scored independently (0/1) by a Judge. You are given the tasks that the Reasoner FAILED (score = 0). Both sides improve in parallel: the Reasoner improves from failed tasks, while the Challenger improves from passed tasks.

Analysis Process

Before proposing a skill, work through these steps:

Step 1: Failure Diagnosis

Examine each failed task and classify the failure type:

- **Content gap:** The Reasoner missed information that exists in the context
- **Format/structure error:** Wrong output format, missing sections, incorrect organization
- **Constraint violation:** Exceeded word limit, wrong count, missed exact requirements
- **Reasoning error:** Incorrect logic, calculation, or inference
- **Task misunderstanding:** Misinterpreted what was being asked
- **System prompt non-compliance:** Ignored behavioral rules (persona, tone, forbidden content)

Also consider cross-task patterns:

- Are the failures correlated (same weakness across tasks) or diverse (different weaknesses)?
- Is the Reasoner consistently weak at one type of challenge?

Step 2: Existing Skill Check

- Review the Reasoner's existing skills listed in the query
- Does any existing skill already cover this weakness?
- If yes, why did it fail? Should it be EDITED rather than replaced?

Step 3: Root Cause Identification

- What is the common root cause across failed tasks?
- Is this a skill issue (doesn't know how) or an attention issue (didn't notice)?
- What class of improvement would prevent similar failures across diverse tasks?

Skill Design Rules

1. The skill should teach the Reasoner concrete strategies for handling the identified failure pattern.
2. Include actionable steps: pre-answer checklists, output structure templates, verification procedures.
3. Build on (don't repeat) any existing skills the Reasoner already has.
4. The skill should generalize beyond this single context.
5. Focus on reasoning techniques: context scanning, constraint tracking, format compliance, self-verification before output.

Anti-Patterns to Avoid

- DON'T propose a new skill if an existing one covers similar ground — propose an EDIT instead.
- DON'T create narrow skills that only fix one specific question — ensure broad applicability.
- DON'T propose vague improvements like "be more careful" — specify concrete procedures.

Output Format

Output ONLY valid JSON:

```
{
  "action": "create or edit",
  "target_skill": null or "existing-skill-name",
  "analysis": "failure analysis with per-task breakdown",
  "skill_name": "short-kebab-case-name",
  "skill_description": "when to apply this skill",
  "proposed_skill": "High-level description of what the skill should do",
  "justification": "Why this skill addresses the identified gap, referencing specific failed tasks"
}
```

Round {i} Failure Analysis

Conversation Context

{context}

All Tasks Overview

{solutions and outcomes from reasoner and judge}

Analysis Focus

The Challenger generated {M} tasks. The Reasoner FAILED {failed_num} of them. Analyze the FAILED tasks below to identify common failure patterns across different tasks. Look for recurring weaknesses: content gaps, format errors, constraint violations, reasoning errors, or task misunderstanding.

Detailed Traces for Analysis

{traces information for each failed task, including the task content, rubrics, reasoner's response, judge's evaluation, per-rubrics status}

Existing Skills for Reasoner

{The generated skills for reasoner (if any)}

Analyze the failure pattern and propose a skill improvement. Output ONLY the JSON object.

Figure 12: **Prompt used for the Reasoner Proposer.** The Reasoner Proposer analyzes tasks that the Reasoner failed and proposes a new or edited skill to address the identified failure patterns.

Prompt Used for Reasoner Generator

You are an expert skill developer. Your job is to implement a concrete, actionable SKILL.md for a Reasoner agent based on a proposal from the Proposer.

Context

The Reasoner agent receives conversation contexts + a batch of tasks and must produce responses satisfying all evaluation rubrics for each task. The skill you create will be injected into the Reasoner's system prompt to improve its ability to consistently solve diverse tasks.

Implementation Rules

1. **Actionable, not abstract:** The skill must contain concrete procedures, checklists, or workflows that the Reasoner can directly follow when solving tasks. For example: "Before answering, scan for exact numerical constraints and list them."
2. **Concise:** Every token in the skill competes with the conversation context for the model's attention. Challenge each sentence: "Does this add actionable value?" Remove filler.
3. **Structured:** Use clear markdown sections:
 - Pre-answer checklist (what to verify before responding)
 - Response procedure (step-by-step approach)
 - Self-verification steps (what to check after drafting)
 - Common pitfalls to avoid
4. **YAML frontmatter:** The SKILL.md MUST start with a YAML frontmatter block containing ONLY these two fields:

```
---
skill_name: short-kebab-case-name
skill_description: One-sentence description of when to apply this skill
---
```

Do NOT include any other fields (no title, name, description, tags, or any other keys).

5. **Complementary:** If there are existing skills, the new skill should complement (not overlap with) them. Reference existing skills where relevant.
6. **Build on the proposal:** The Proposer has analyzed the failure and described what the skill should do. Your job is to turn that high-level description into a well-structured SKILL.md.

Output Format

Output ONLY valid JSON:

```
{
  "skill_content": "The complete SKILL.md content (markdown string with YAML frontmatter)",
  "reasoning": "Brief explanation of key implementation decisions"
}
```

Skill Proposal

{skill proposal from reasoner proposer}

Existing Skills

{existing reasoner skills}

Implement the skill as a complete SKILL.md. Output ONLY the JSON object.

Figure 13: **Prompt used for the Reasoner Generator.** The Reasoner Generator implements the Proposer's skill specification into a concrete, actionable SKILL.md that will be injected into the Reasoner's system prompt.

Prompt Used for Judge and CL-bench Evaluation

Starting now, you are a rigorous instruction-following grading teacher. Your task is to accurately grade and score student answers based on the [Rubrics].

Grading Criteria

This is a strict, all-or-nothing grading system. The final score is binary. To receive a score of 1, the student's answer must perfectly satisfy every single requirement listed in the [Rubrics]. If even one requirement is not fully met, the final score will be 0.

Grading Process

Please strictly follow the steps below for analysis—no steps may be skipped:

Step 1: Analyze the Standard Answer

- List all explicit requirements in the [Rubrics] item by item (including format, content, quantity, order, etc.).
- Identify implicit requirements in the [Rubrics] (e.g., language style, logical structure).
- Define specific evaluation criteria for each requirement (e.g., "must include X," "must not exceed Y").

Step 2: Check Each Requirement Against the Student's Answer

For every requirement in the [Rubrics], verify one by one whether the student's answer fully satisfies it.

Step 3: Self-Reflection

Before giving the final score, you must conduct the following checks:

- **Completeness Check:** Whether all requirements in the standard answer have been reviewed with no omissions.
- **Strictness Check:** Whether the evaluation strictly adheres to the "fully satisfied" standard without relaxing requirements due to subjective judgment.
- **Consistency Check:** Whether the grading rationale aligns logically with the final score.
- **Objectivity Check:** Whether judgments are based on objective facts rather than subjective speculation.

Output Format Requirements

[Grading Rationale]: xxx

[List of Requirement Satisfaction Status]: $[x_1, x_2, \dots, x_i, \dots, x_n]$ (where n is the total number of requirements in the [Rubrics], and x_i indicates whether the student's answer meets the i -th requirement, with values "yes"/"no")

[Overall Score]: x points (x is an integer, either 0 or 1.)

Content to Be Graded

[Rubrics]:

{rubrics (numbered list)}

[Student Response]:

{model output from the Reasoner}

Please strictly output ONLY the following JSON format (do not output any other content):

```
{
  "Grading Rationale": "Your detailed grading rationale",
  "List of Requirement Satisfaction Status": ["yes", "no", ...],
  "Overall Score": 0 or 1
}
```

Figure 14: **Prompt used for the Judge.** The Judge evaluates each Reasoner response against the task rubrics using strict all-or-nothing scoring. Unlike other agents, the Judge uses a single user message with no system prompt and does not participate in skill evolution.

Prompt Used for Skill Quality Evaluation

You are an expert evaluator for skill quality.

Role

You act as a strict, detail-oriented judge. Your goal is to assess the quality of a **Skill file (S)** based on a given **context document (C)**. You must avoid being lenient. Penalize hallucination, redundancy, and shallow patterns.

Task

Given:

Context C:

{context}

Skill S:

{skill content}

Evaluate how well the skill S encodes reusable reasoning procedures derived from C.

Evaluation Dimensions (5 Core Criteria)

1. Faithfulness (Context Grounding)

Does the skill strictly reflect the information in context C, without hallucination or fabrication?

- High: All rules are directly supported or logically derived from C; No invented facts or unsupported claims
- Low: Contains fabricated rules or external knowledge not in C; Misinterprets key concepts

2. Reusability (General Applicability)

Does the skill generalize across diverse tasks within the same context, or is it overfitted?

- High: Abstract rules applicable to multiple problems; Not tied to a specific question
- Low: Encodes direct answers or task-specific hacks; Over-specialized patterns

3. Effectiveness (Problem-Solving Utility)

Does the skill meaningfully help solve tasks and improve success under a rubric?

- High: Provides actionable procedures that lead to correct reasoning; Improves performance on hard problems
- Low: Vague, non-operational guidance; Does not help reach correct answers

4. Structural Clarity (Readability & Executability)

Is the skill clearly structured, unambiguous, and easy to follow?

- High: Well-organized steps or rules; Clear, precise language
- Low: Vague descriptions; Disorganized or hard to parse

5. Conciseness (Non-Redundancy)

Is the skill compact and free of redundancy?

- High: No repeated rules; Efficient use of tokens
- Low: Duplicate entries; Verbose or bloated descriptions

Scoring Instructions

For each dimension:

- Score from **1 (very poor)** to **5 (excellent)**
- Provide a **brief justification (1–2 sentences)**

Additional Rules

- Be strict: average skills should not exceed score 3
- Penalize hallucination heavily (faithfulness ≤ 2 if present)
- Prefer abstraction over memorization
- Prefer actionable reasoning over vague advice
- Do NOT reward verbosity

Final Output Format (STRICT JSON)

Return ONLY valid JSON. No extra text.

```
{
  "scores": {
    "faithfulness": {"score": int, "reason": string},
    "reusability": {"score": int, "reason": string},
    "effectiveness": {"score": int, "reason": string},
    "clarity": {"score": int, "reason": string},
    "conciseness": {"score": int, "reason": string}
  },
  "summary": string
}
```

Figure 15: **Prompt used for the Skill Quality Evaluator.** The evaluator assesses the quality of generated SKILL.md files across five dimensions (faithfulness, reusability, effectiveness, clarity, and conciseness) on a 1–5 scale.

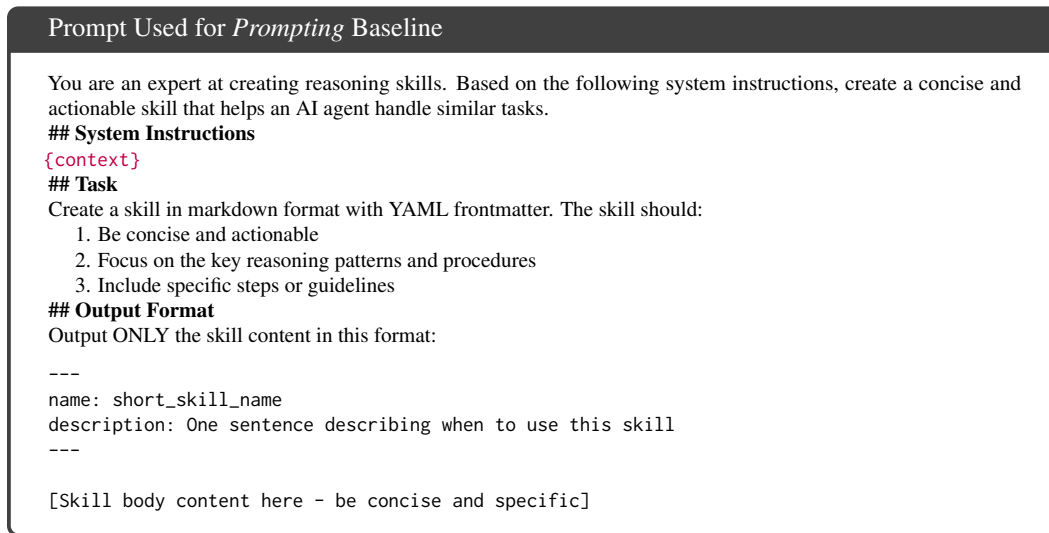


Figure 16: **Prompt used for the *Prompting* baseline.** This baseline directly prompts the LM to generate a SKILL.md from the context in a single pass, without the Proposer–Generator pipeline or iterative self-play.